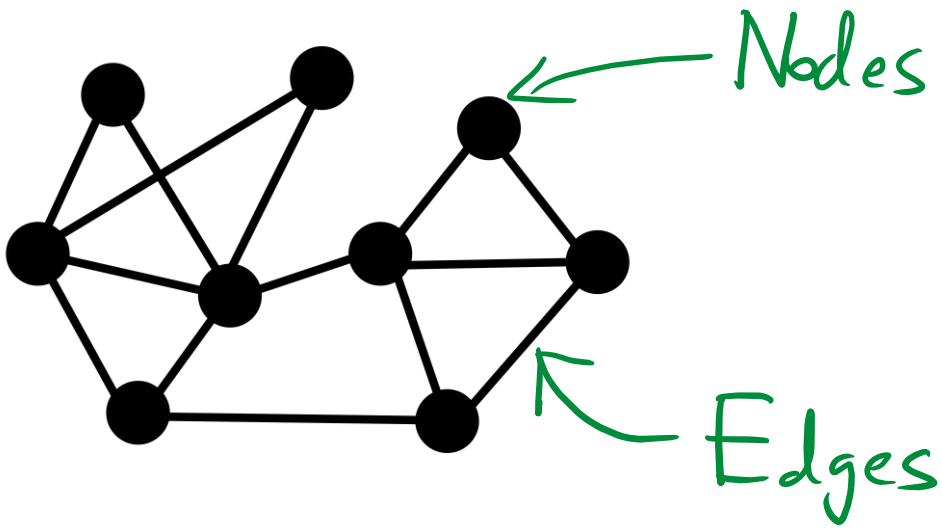
Lecture 8 – Graph Neural Networks

Sanka Mohottala

Definitions

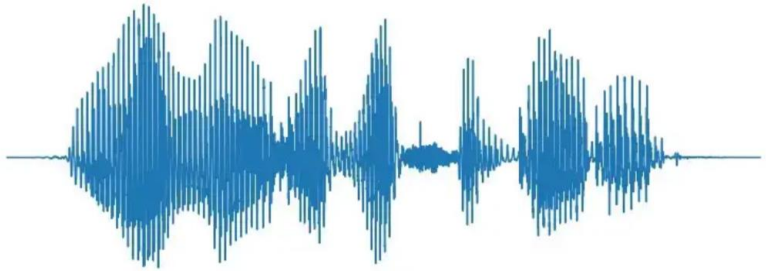
- Graphs are a general language for describing and analyzing entities with relations/interactions.
- Networks are real-world instantiations of graphs with node and/or edge features.



- **Objects:** nodes, vertices — N
- **Interactions:** links, edges — E
- **System:** network, graph — $G(N,E)$

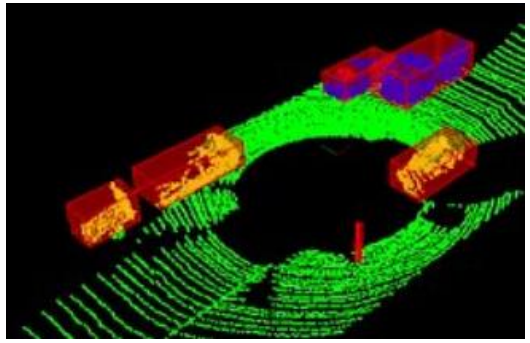
Why GNNs?

- Traditionally , deep learning deals with Euclidian data such as grids, sequences etc.

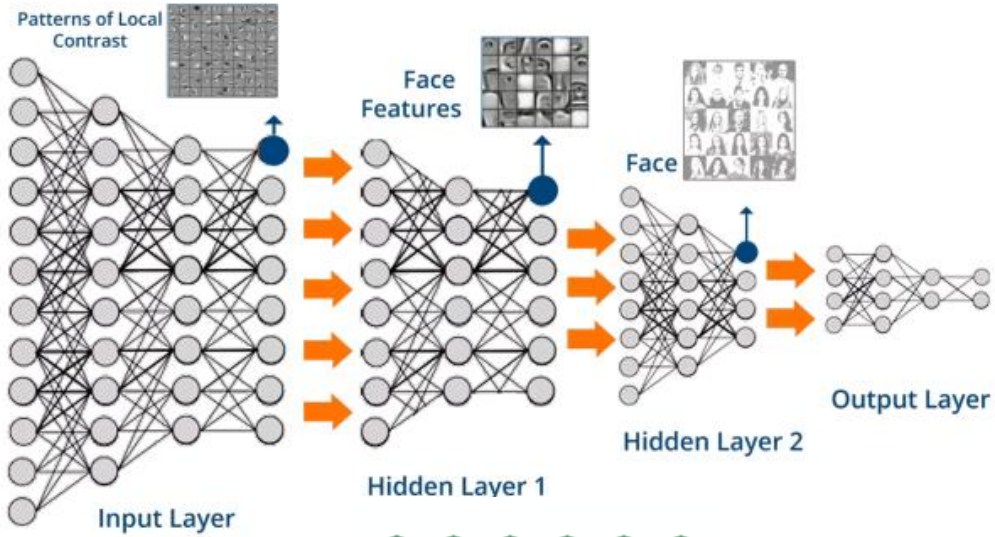


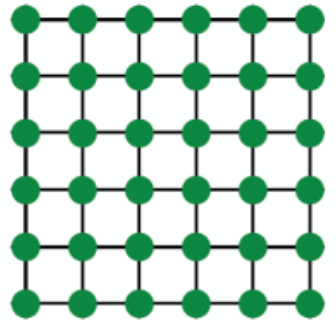
$\mathbb{R} \mapsto \mathbb{R}$: Signal , Speech etc.

$\mathbb{R}^2 \mapsto \mathbb{R}$: Images



$\mathbb{R}^3 \mapsto \mathbb{R}^n$: Point clouds, Volumes

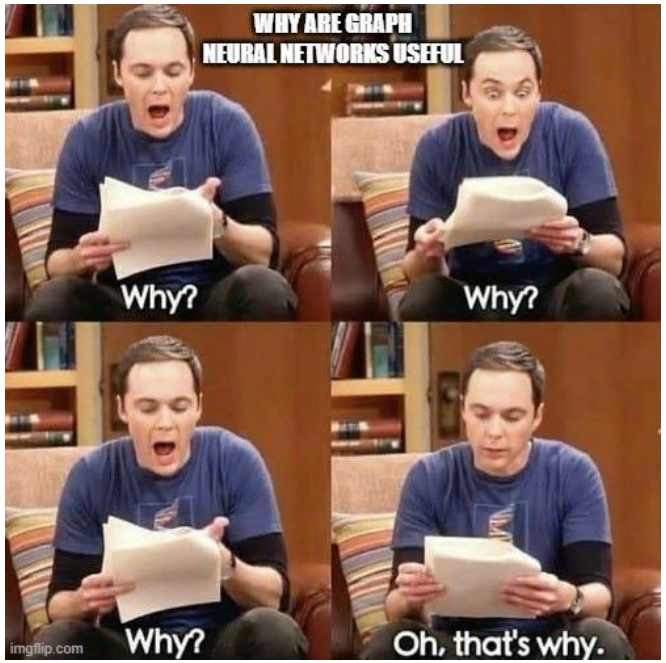


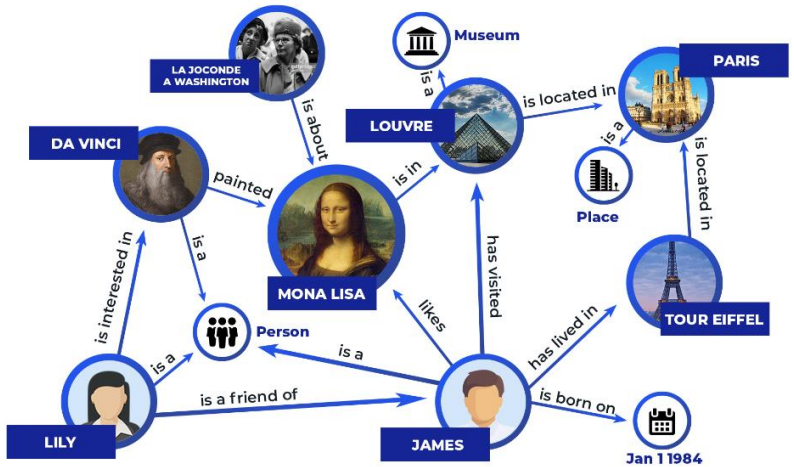


Images

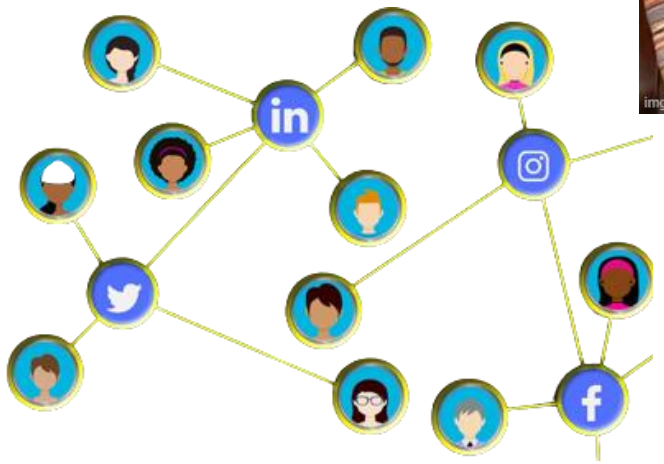
Networks are complex.

- Arbitrary size and complex topological structure (*i.e.*, no spatial locality like grids)





Knowledge Graph

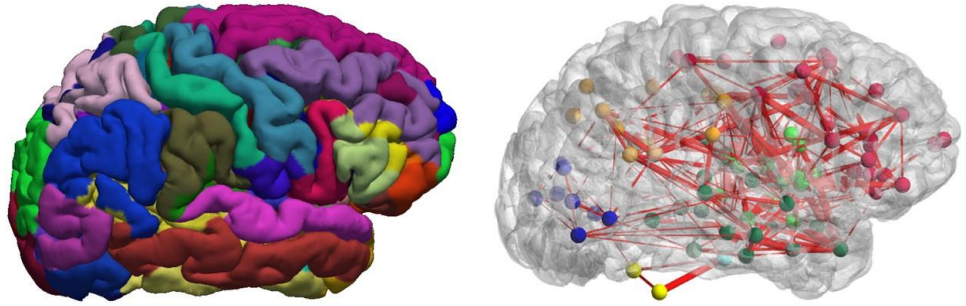


Social Networks

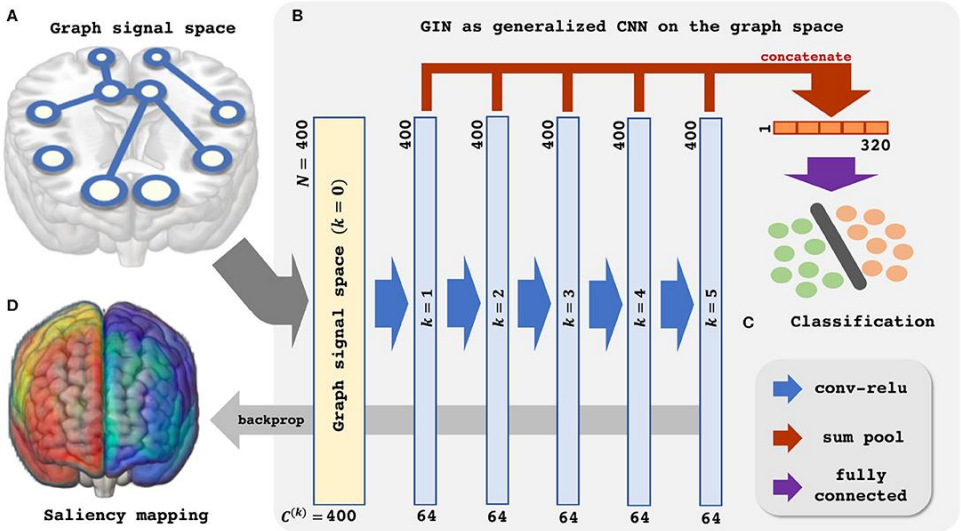
- No fixed node ordering or reference point
- Often dynamic and have multimodal features

Graph neural network applications

- Graphs can be used to model brain connectomes taken from fMRI/DTI data



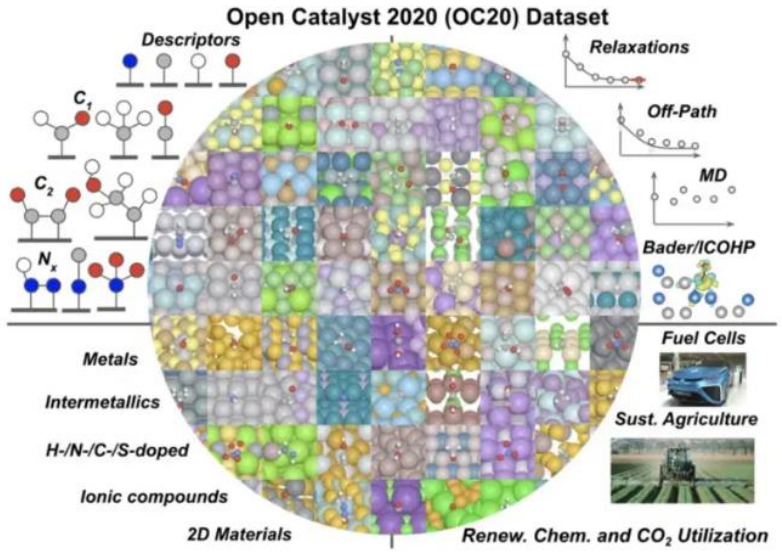
- GNN can be used for fMRI data analysis by modeling the functional connectivity of brain as a graph structure.

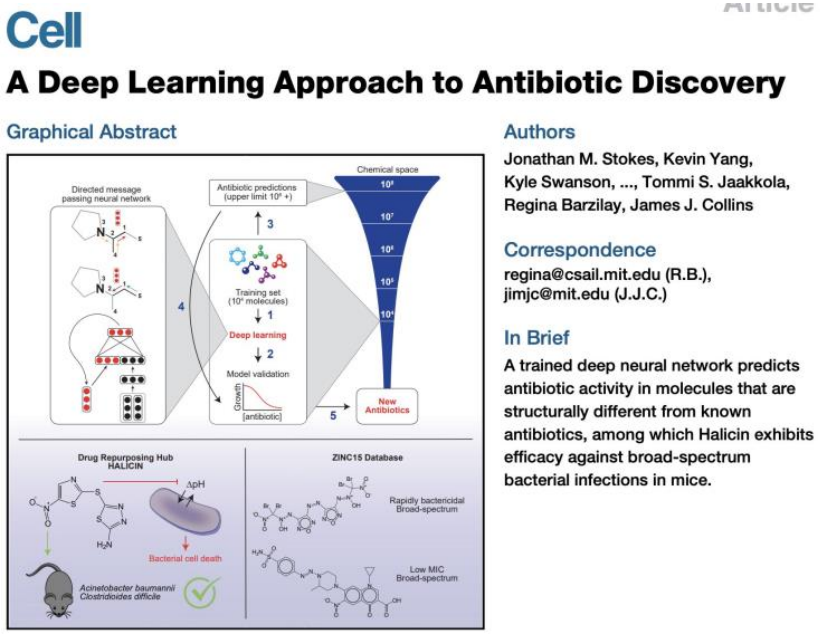


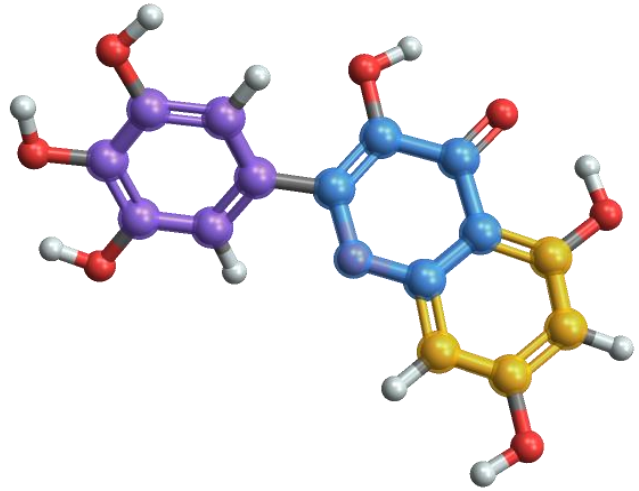
Graphs are also used in Chemistry and Pharmacology

Modeling molecules as graphs data, they can be used with graph neural networks to do

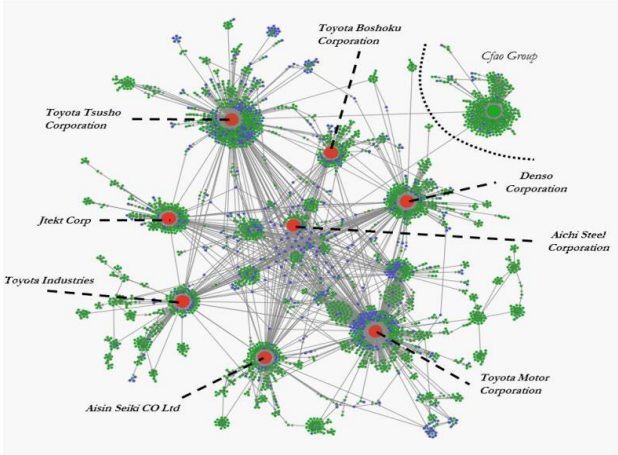
- Molecule Analysis
- Drug discovery
- Drug repurposing
- Catalysts discovery



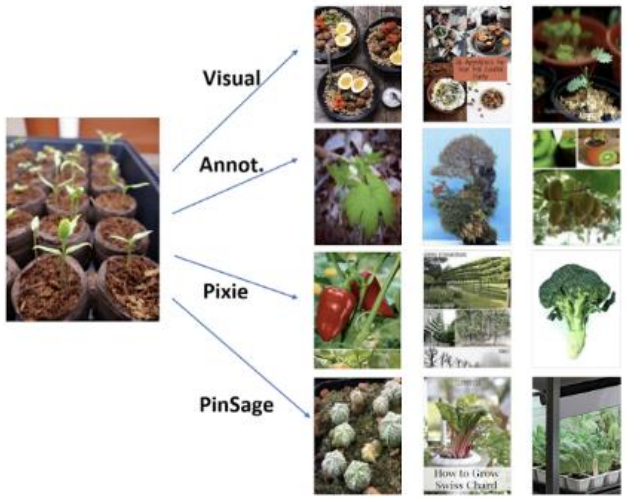


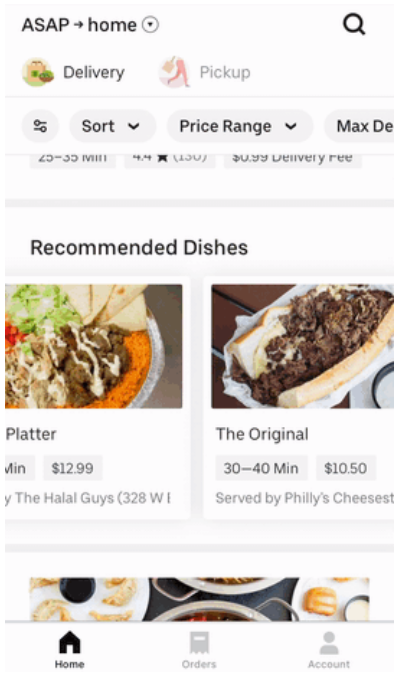


In financial sector, graphs can be used to model how the markets work as in the case of stock market prediction.



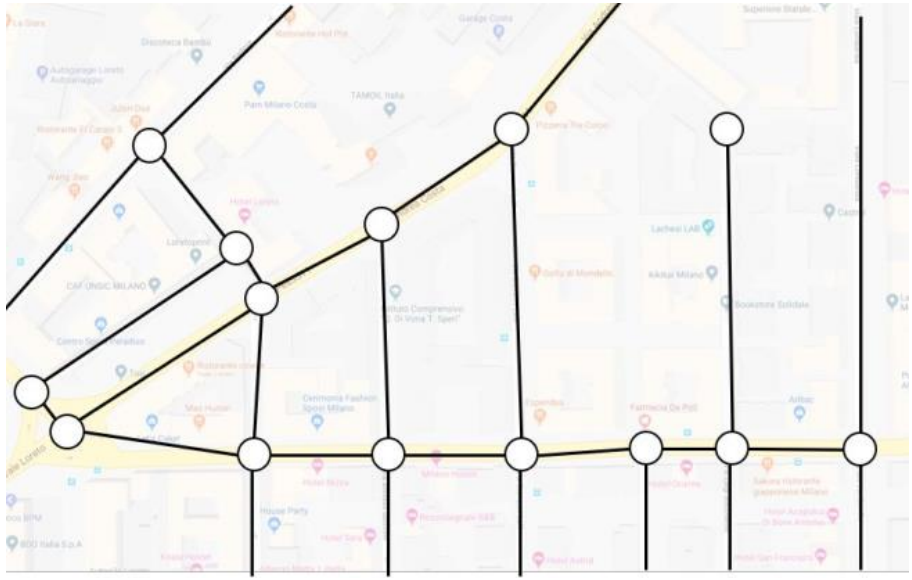
Recommender systems used in Uber Eats and Pinterest uses GNN algorithms.

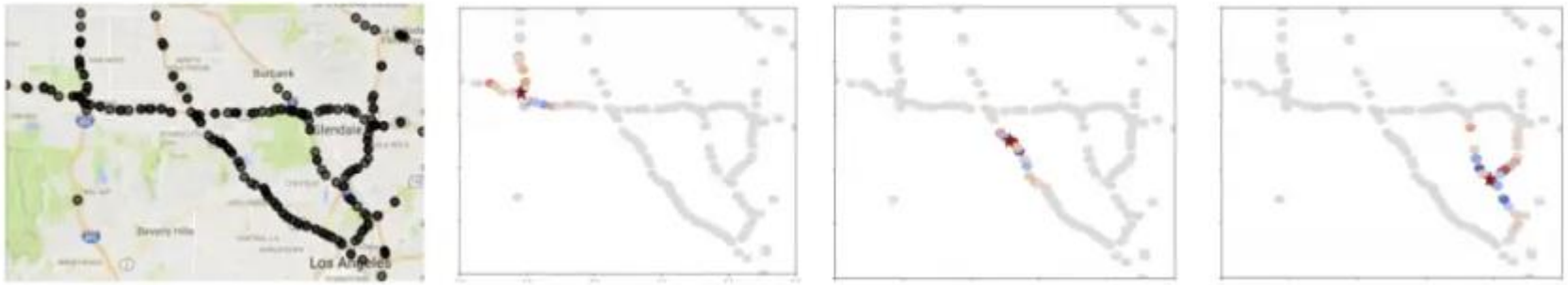


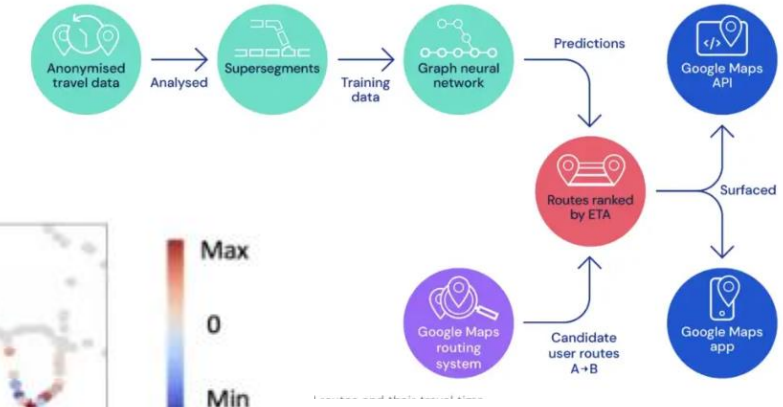


Road systems can also be modeled as graph. In such a system, intersections can be considered as nodes and roads as edges.

GNNs are being used with such data to estimate the time of arrival in Google maps.







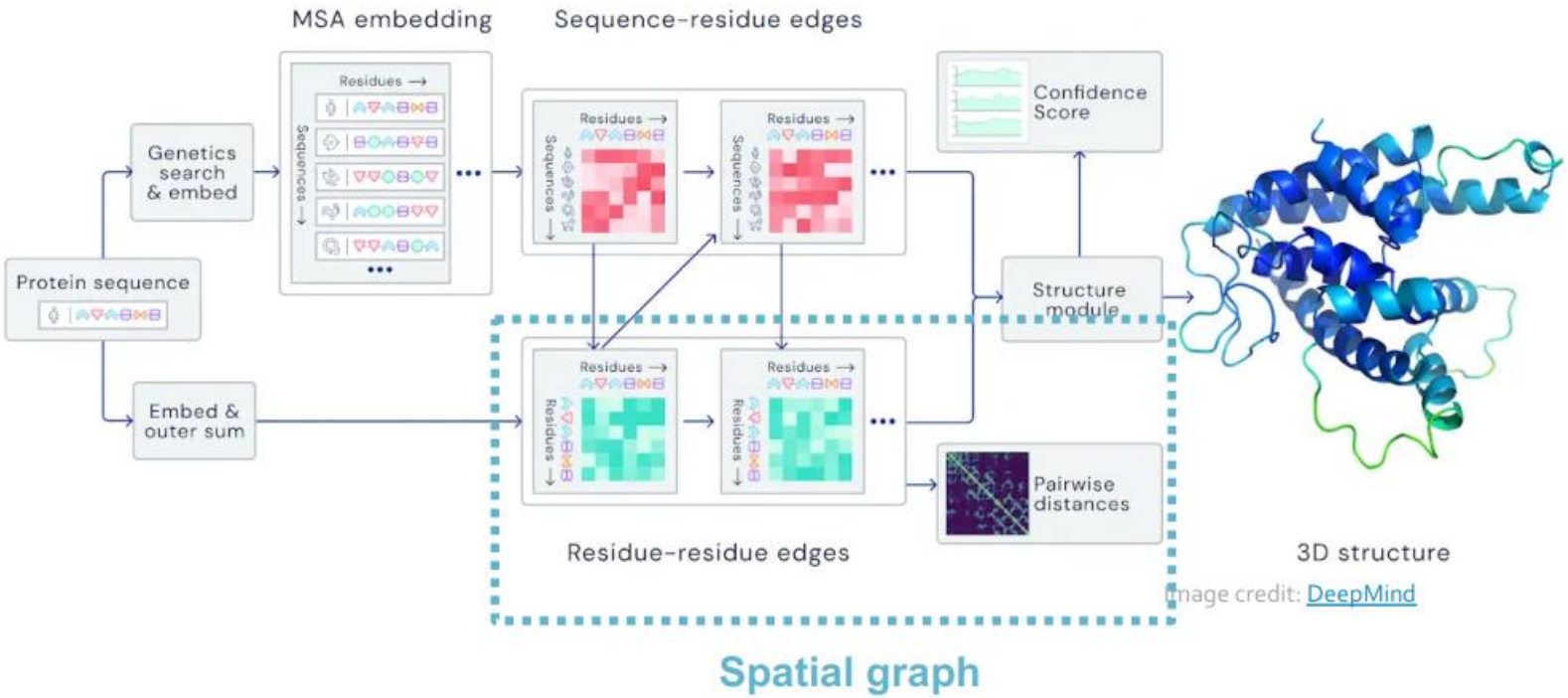
Graph ML Tasks

- **Node classification**: Predict a property of a node
  - **Example:** Categorize online users / items
- **Link prediction**: Predict whether there are missing links between two nodes
  - **Example:** Knowledge graph completion
- **Graph classification**: Categorize different graphs
  - **Example:** Molecule property prediction
- **Clustering**: Detect if nodes form a community
  - **Example:** Social circle detection
- **Other tasks**:
  - **Graph generation**: Drug discovery
  - **Graph evolution**: Physical simulation

Node-level Tasks

Computationally predict a protein's 3D structure based solely on its amino acid sequence

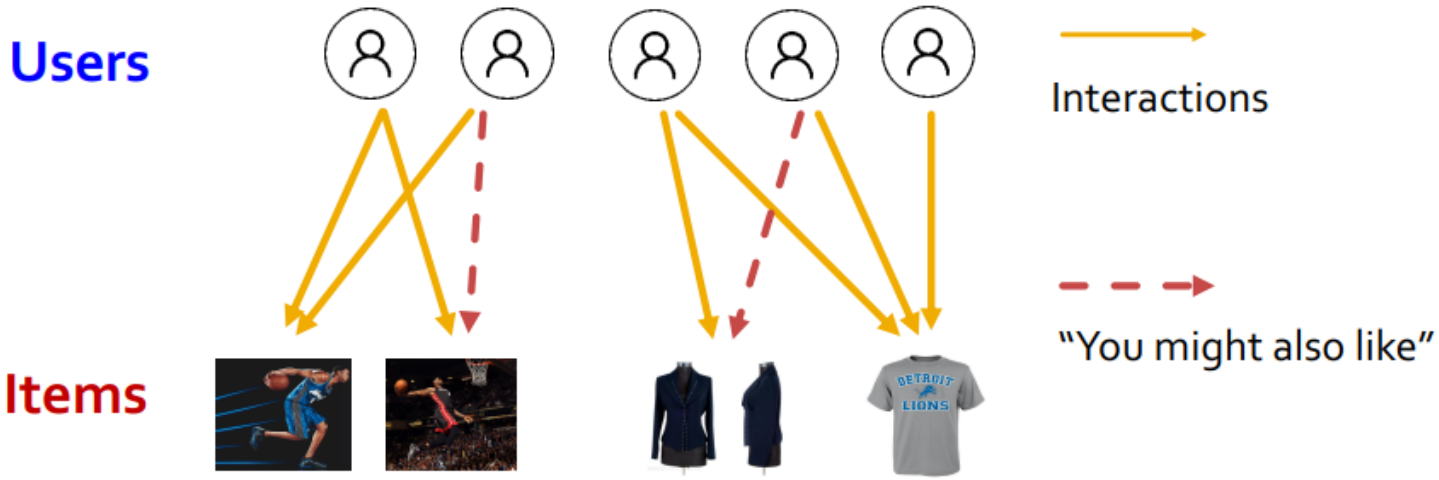
- **Key idea:** "Spatial graph"
  - **Nodes:** Amino acids in a protein sequence
  - **Edges:** Proximity between amino acids (residues)



Edge-level Task

Recommender Systems

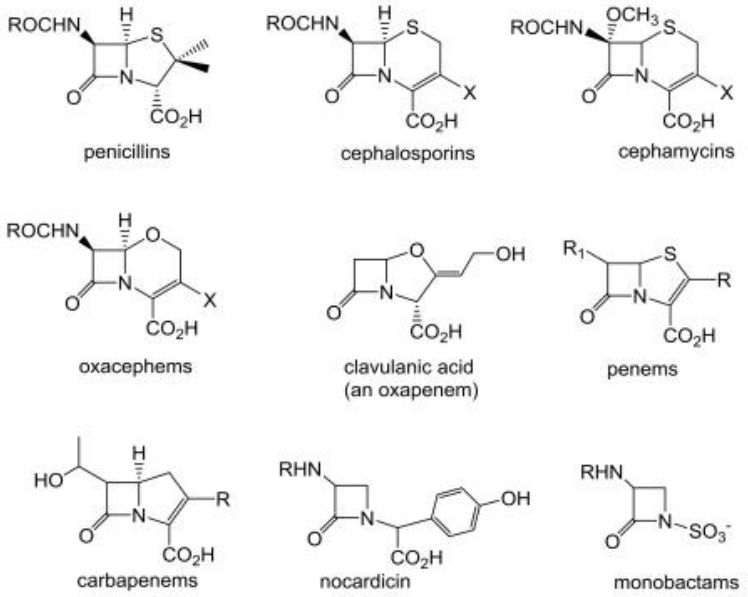
- **Users interacts with items**
  - Watch movies, buy merchandise, listen to music
  - **Nodes:** Users and items
  - **Edges:** User-item interactions
- **Goal: Recommend items users might like**



Graph-level Task

Drug discovery

- **Antibiotics are small molecular graphs**
  - **Nodes:** Atoms
  - **Edges:** Chemical bonds

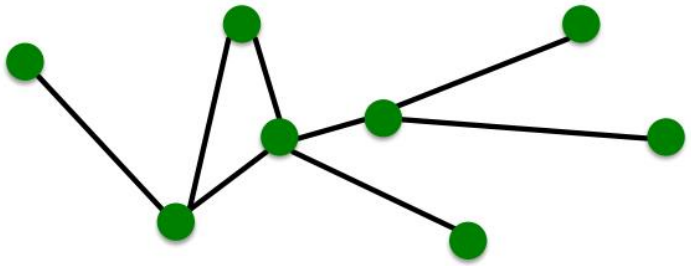


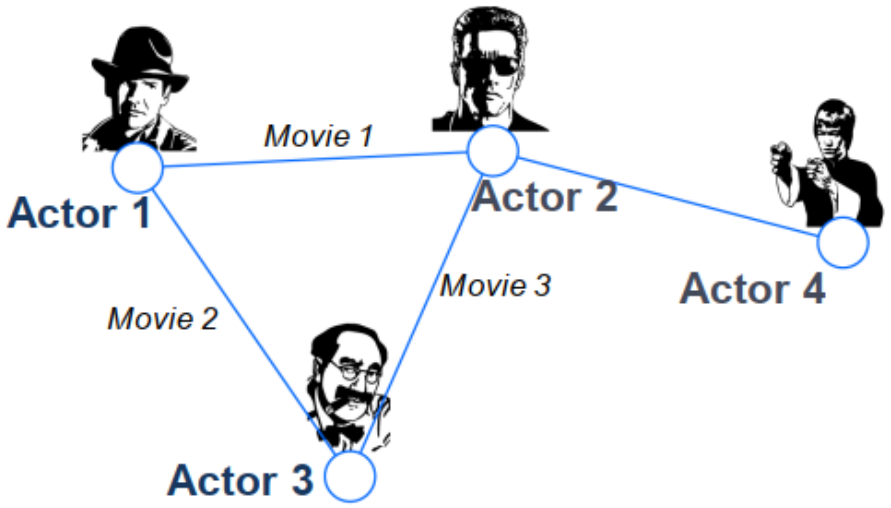
Konaklieva, Monika I. "Molecular targets of β-lactam-based antimicrobials: beyond the usual suspects." Antibiotics 3.2 (2014): 128-142.

Image credit: CNN

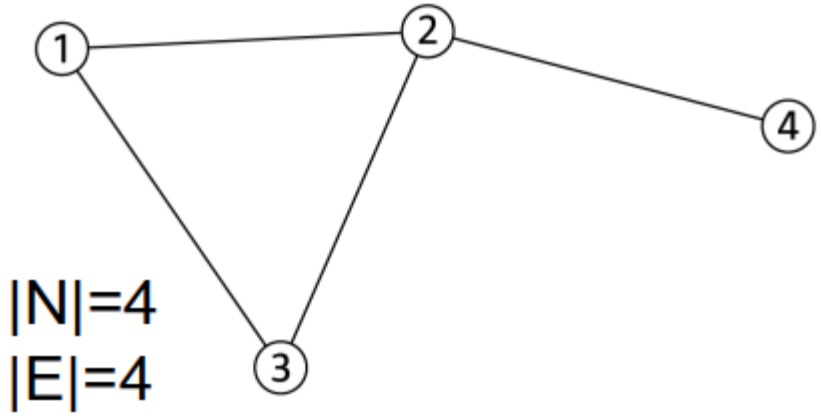
Basic Graph Theory Concepts

Components of a Network





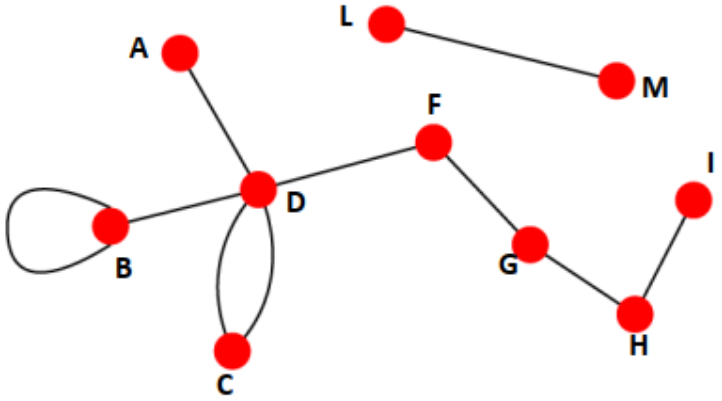
- **Objects:** nodes, vertices — N
- **Interactions:** links, edges — E
- **System:** network, graph — $G(N,E)$



Directed vs Undirected Graphs

Undirected

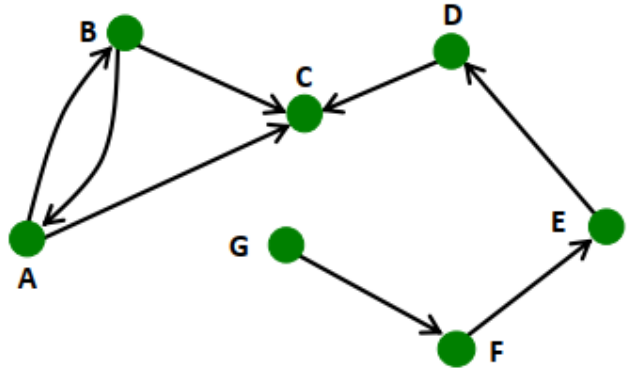
- **Links:** undirected (symmetrical, reciprocal)



- **Examples:**
  - Collaborations
  - Friendship on Facebook

Directed

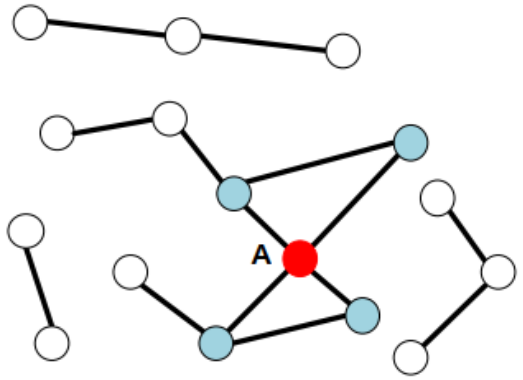
- **Links:** directed (arcs)



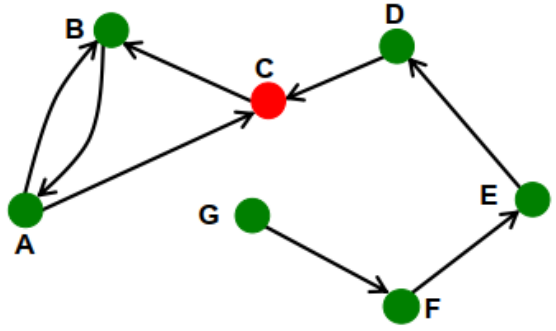
- **Examples:**
  - Phone calls
  - Following on Twitter

Node Degree

Undirected



Directed



Source: Node with $k^{in} = 0$
Sink: Node with $k^{out} = 0$

Node degree, k_i: the number of edges adjacent to node ***i***

$$k_A = 4$$

Avg. degree: $\bar{k} = \langle k \rangle = \frac{1}{N}\sum_{i=1}^{N} k_i = \frac{2E}{N}$

In directed networks we define an **in-degree** and **out-degree.**

The (total) degree of a node is the sum of in- and out-degrees.

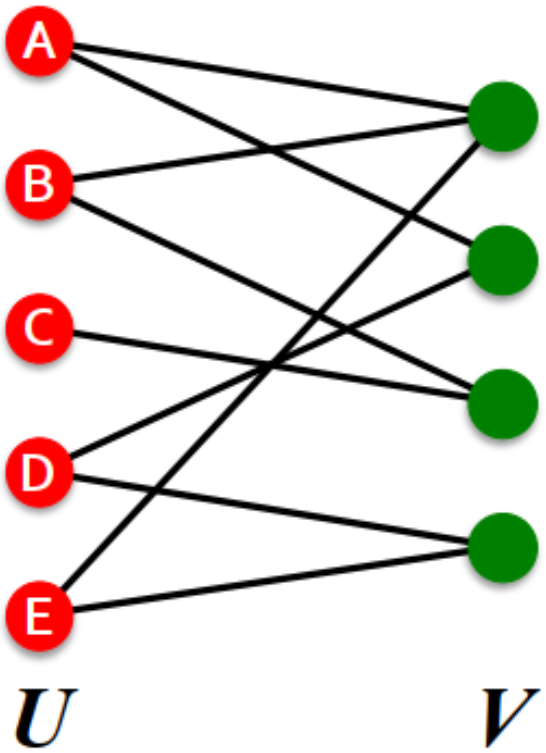
$$k_C^{in} = 2 \qquad k_C^{out} = 1 \qquad k_C = 3$$

$$\bar{k} = \frac{E}{N} \qquad \overline{k^{in}} = \overline{k^{out}}$$

Bipartite Graph

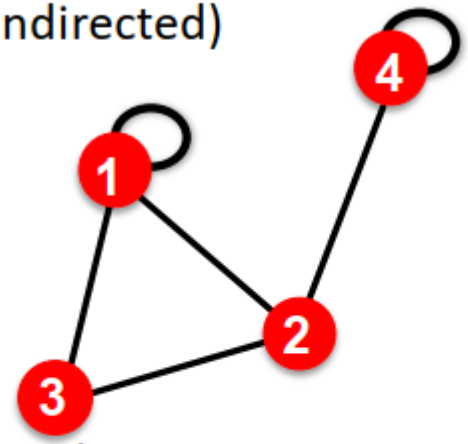
- **Bipartite graph** is a graph whose nodes can be divided into two disjoint sets ***U*** and ***V*** such that every link connects a node in ***U*** to one in ***V***; that is, ***U*** and ***V*** are **independent sets**

- **Examples:**
  - Authors-to-Papers (they authored)
  - Actors-to-Movies (they appeared in)
  - Users-to-Movies (they rated)
  - Recipes-to-Ingredients (they contain)



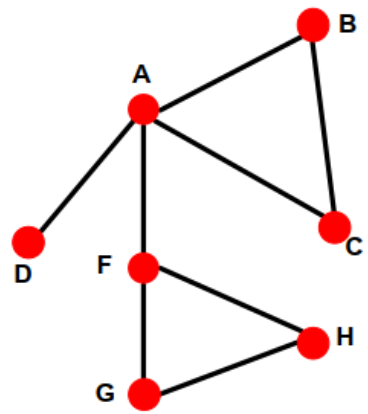
- **Self-edges (self-loops)**

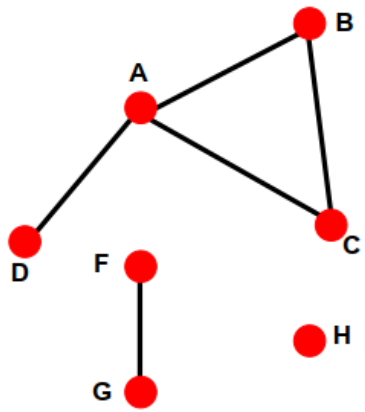
(undirected)



$$A_{ij} = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix}$$

- **Connected (undirected) graph:**
  - Any two vertices can be joined by a path
- A disconnected graph is made up by two or more connected components

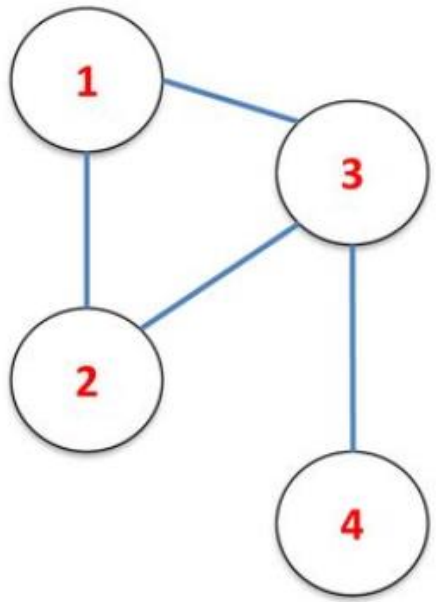




Largest Component:
Giant Component

Isolated node (node H)

Graph Representation : Matrix form



Example graph

Adjacency matrix(A)

	1	2	3	4
1	0	1	1	0
2	1	0	1	0
3	1	1	0	1
4	0	0	1	0

Degree matrix (D)

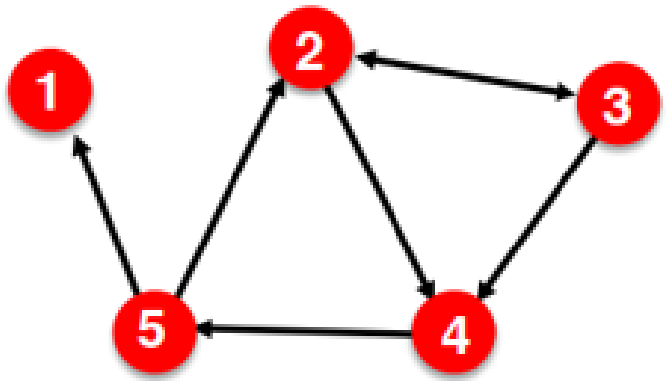
	1	2	3	4
1	2	0	0	0
2	0	2	0	0
3	0	0	2	0
4	0	0	0	1

Laplacian ($L=D-A$)

	1	2	3	4
1	2	-1	-1	0
2	-1	2	-1	0
3	-1	-1	2	-1
4	0	0	-1	1

Graph Representation : Other forms

- **Represent graph as a list of edges:**
  - (2, 3)
  - (2, 4)
  - (3, 2)
  - (3, 4)
  - (4, 5)
  - (5, 2)
  - (5, 1)



- **Adjacency list:**
  - Easier to work with if network is
    - Large
    - Sparse
  - Allows us to quickly retrieve all neighbors of a given node
    - 1:
    - 2: 3, 4
    - 3: 2, 4
    - 4: 5
    - 5: 1, 2

Representational Learning

- One common denominator in all the node-level, edge-level and graph-level tasks is finding representations for nodes or edges or the entire graphs.
- This of course isn’t new to machine learning, after all, machine learning is dominated by representational learning.
- Models such as Convolutional Neural Network(CNN), Word2Vec (Skipgram, CBOW models), transformers have been doing this under the hood.

Images

CNN

0.23
0.345
.
.
.
0.64

Parrot 0.98
Dog 0.01
Cat 0.01

Unstructured data

Embedding Space

Passed into MLP for downstream tasks like classification/regression

Text

"Good"

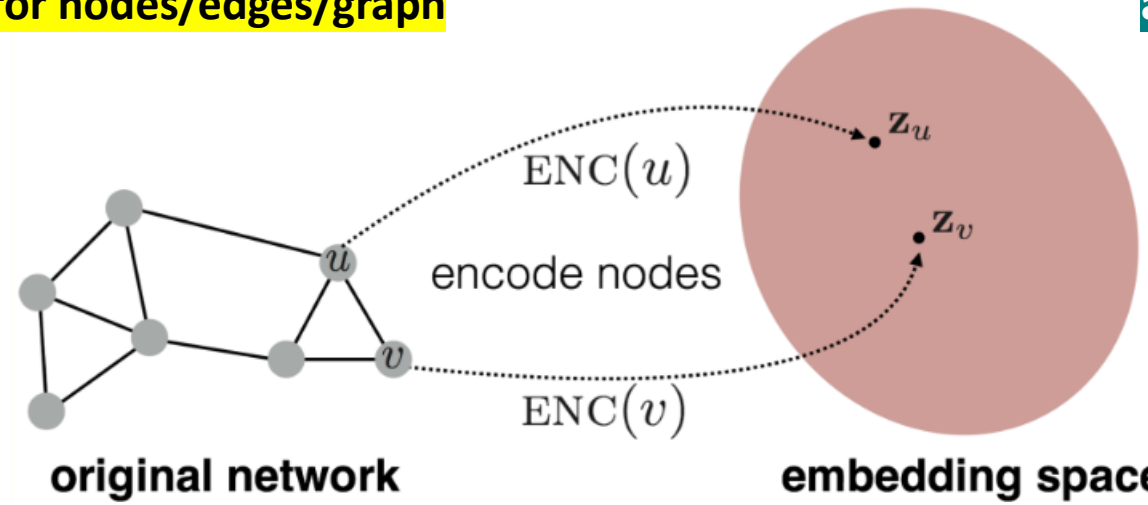
Word2Vec

0.317
-0.45
.
.
.
0.492

Positive 0.98
Negative 0.01
Neutral 0.01

Same is required for Graph Machine Learning and Graph Neural Networks(GNN) is the state-of-the-art appraoch to achieve this.

We have a graph and we find a representation for nodes/edges/graph



Take the embedding, in this case a node embedding

$\boldsymbol{z_u}$

Perform down stream task

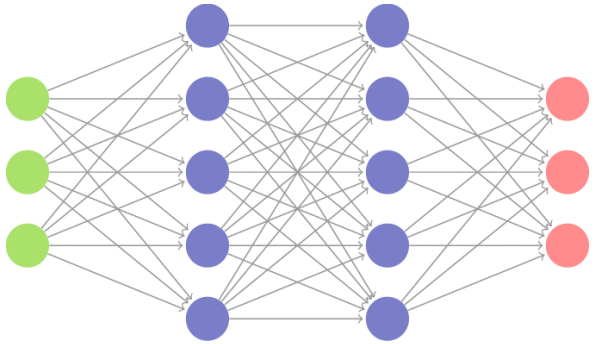


Image source : https://snap-stanford.github.io/cs224w-notes/machine-learning-with-networks/node-representation-learning

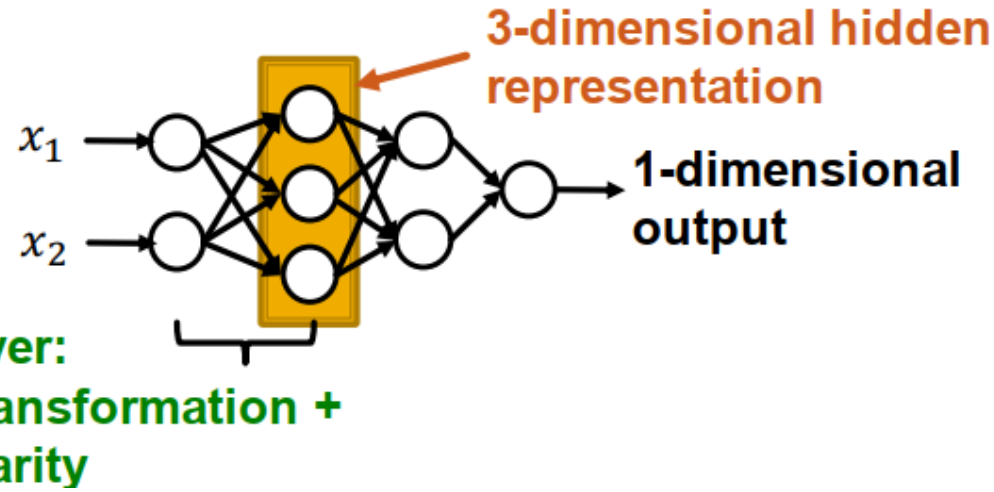
Graph Neural Networks

DL notation recap

- **Each layer of MLP combines linear transformation and non-linearity:**

$$\boldsymbol{x}^{(l+1)} = \sigma(W_l \boldsymbol{x}^{(l)} + b^l)$$

  - where W_l is weight matrix that transforms hidden representation at layer l to layer $l+1$
  - b^l is bias at layer l, and is added to the linear transformation of $\boldsymbol{x}^{(l)}$
  - σ is non-linearity function (e.g., sigmoid)
- Suppose $\boldsymbol{x}$ is 2-dimensional, with entries x_1 and x_2



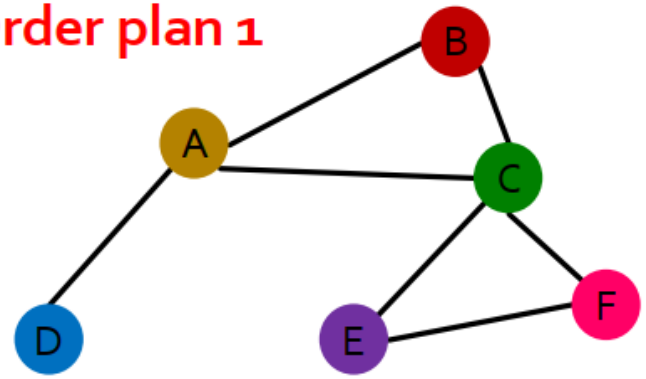
Graph notation recap

- **Assume we have a graph G:**
  - V is the **vertex set**
  - $\boldsymbol{A}$ is the **adjacency matrix** (assume binary)
  - $\boldsymbol{X} \in \mathbb{R}^{|V| \times d}$ is a matrix of **node features**
  - v: a node in V; $N(v)$: the set of neighbors of v.
  - **Node features:**
    - Social networks: User profile, User image
    - Biological networks: Gene expression profiles, gene functional information
    - When there is no node feature in the graph dataset:
      - Indicator vectors (one-hot encoding of a node)
      - Vector of constant 1: [1, 1, …, 1]

Permutation invariance

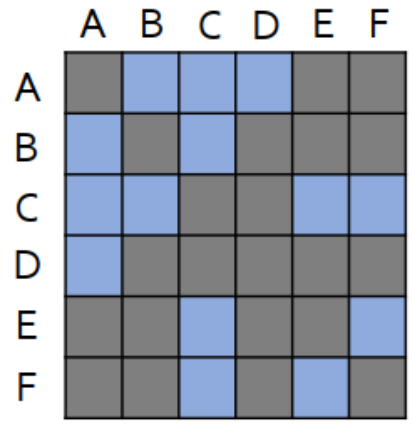
- **Graph does not have a canonical order of the nodes!**

Order plan 1

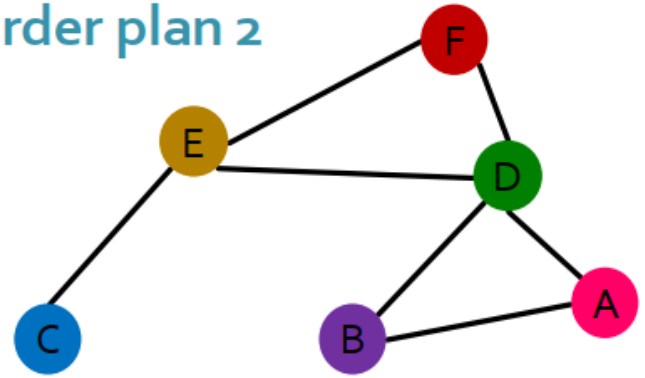


Node features X_1

Adjacency matrix A_1

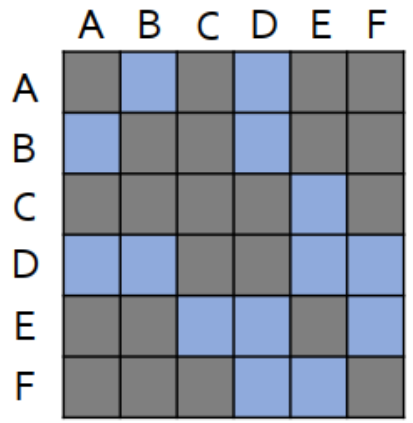


Order plan 2



Node features X_2

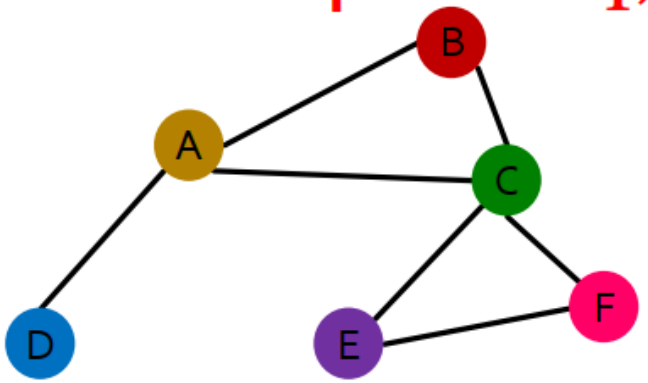
Adjacency matrix A_2



Permutation equivariance

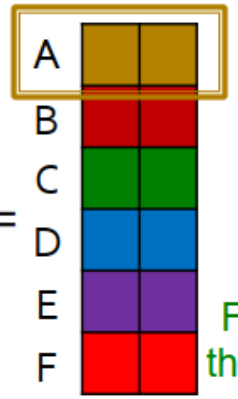
For node representation: We learn a function f that maps nodes of G to a matrix $\mathbb{R}^{m \times d}$.

Order plan 1: A_1, X_1

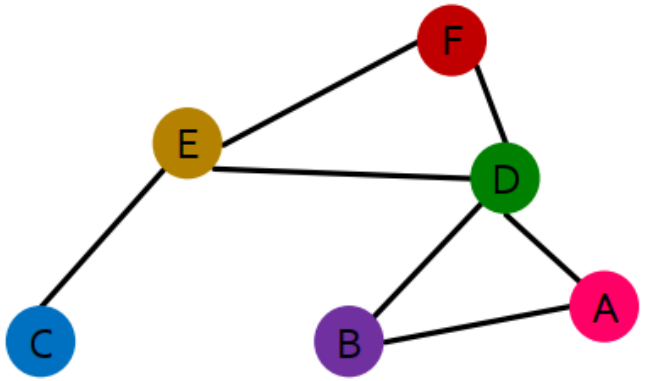


Representation vector of the brown node A

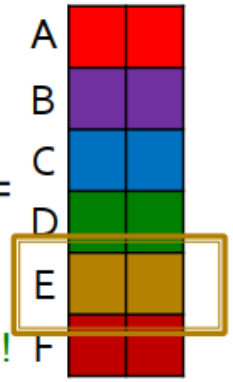
$$f(A_1, X_1) =$$



Order plan 2: A_2, X_2



$$f(A_2, X_2) =$$



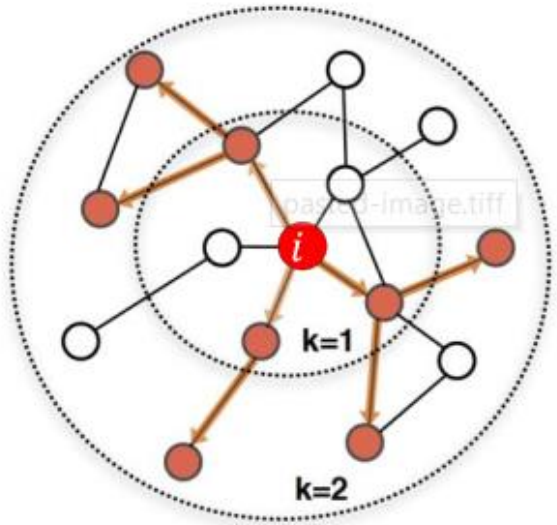
Representation vector of the brown node E

For two order plans, the vector of node at the same position in the graph is the same!

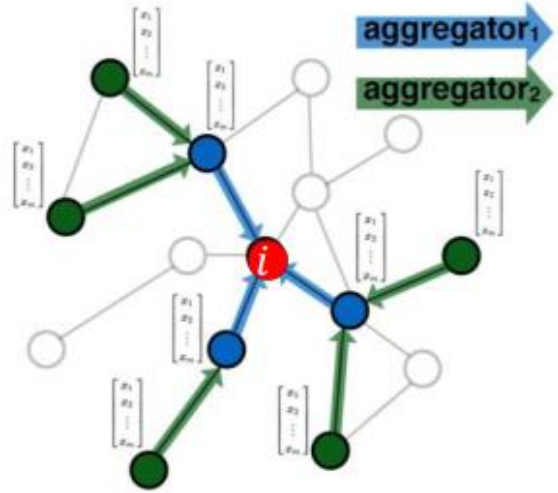
Graph Neural Networks

- GNNs should utilize the bias of graph structures as well as the features present the graph to learn representations.
- Majority of the introduced GNN models follow the 3 building blocks given below,
  - **Message Passing**
  - **Aggregation**
  - **Update**
- Many variants of GNN have been introduced over time, they mainly differ in varying the approach towards one of the 3 mentioned above. Few examples of the variants are Graph Convolution Network (GCN), Graph Sage, Graph Attention Network, Graph Isomorphism Network etc..
- We will address the above-mentioned methodology one by one.
- Here we will take undirected graph with node features only for simplicity. This same method can be extended to tackle edge features or graph level features.

Idea: Node's neighborhood defines a computation graph



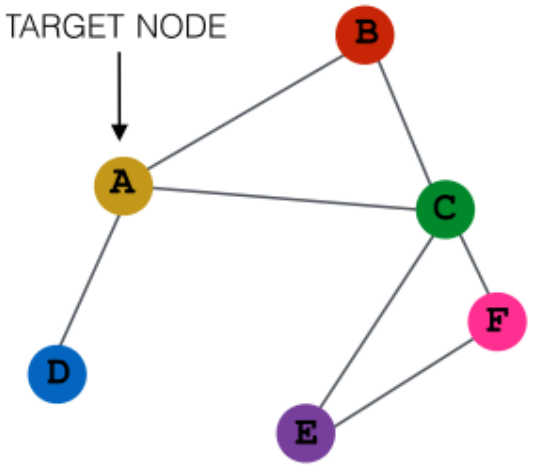
Determine node computation graph



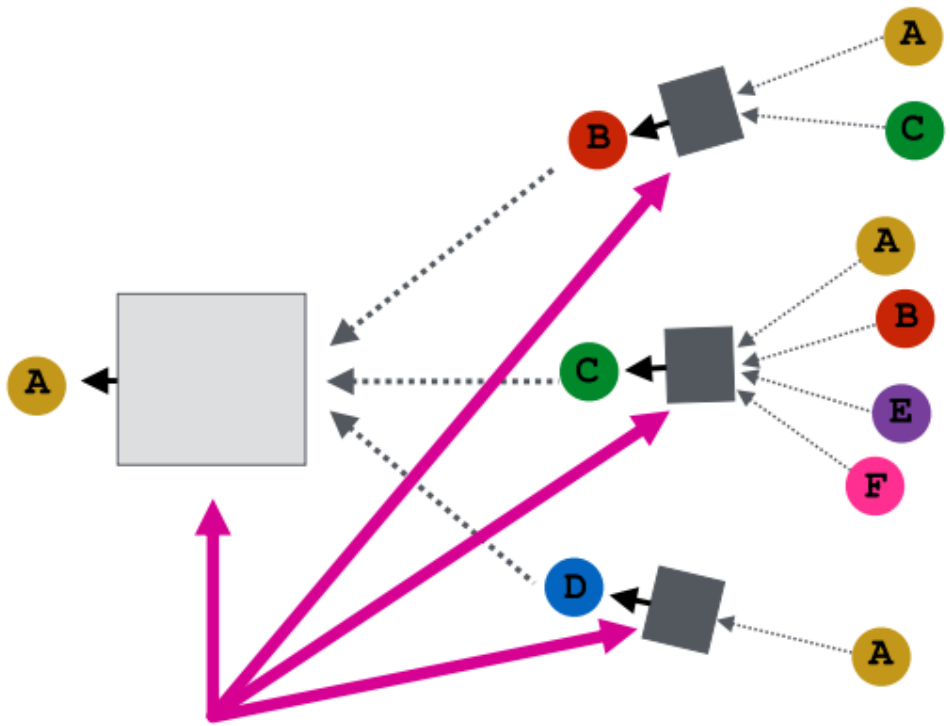
Propagate and transform information

Learn how to propagate information across the graph to compute node features

- **Key idea:** Generate node embeddings based on **local network neighborhoods**
- **Intuition:** Nodes aggregate information from their neighbors using neural networks



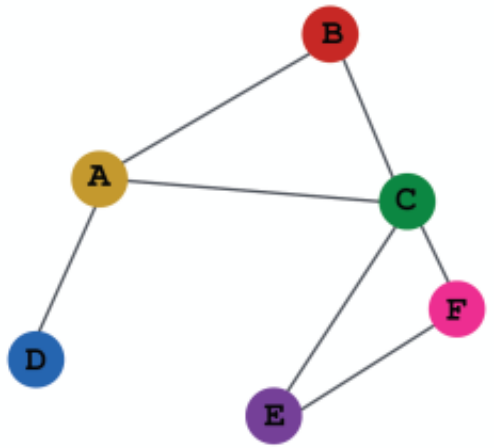
INPUT GRAPH



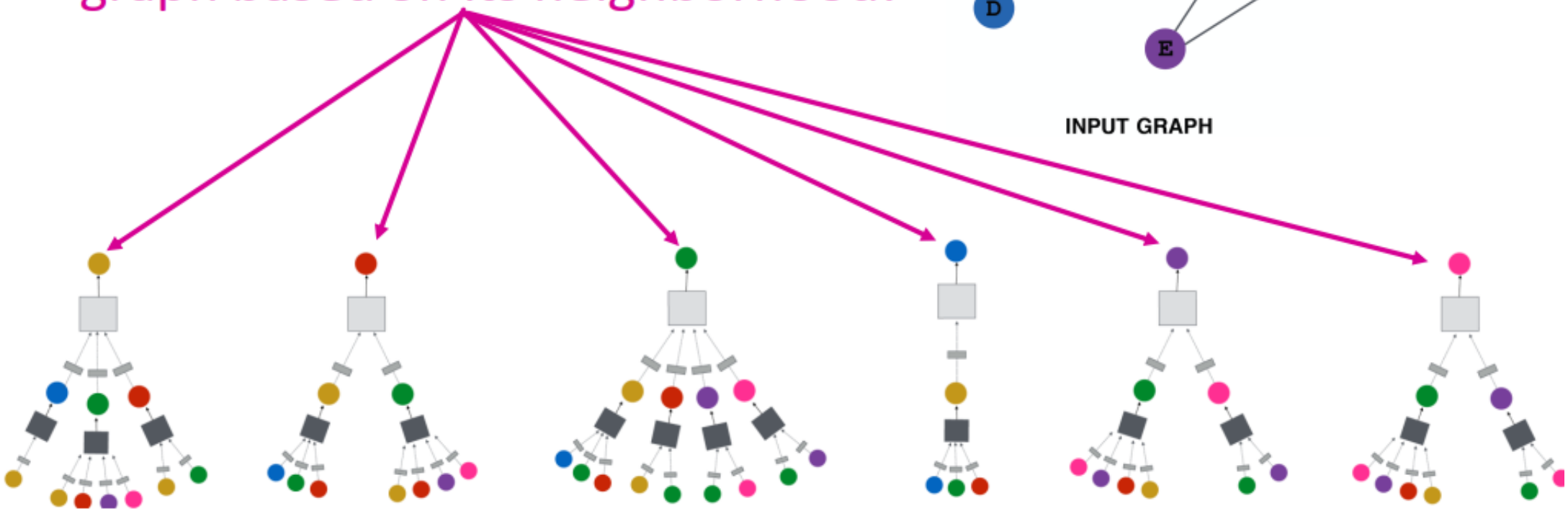
Neural networks

- **Intuition:** Network neighborhood defines a computation graph

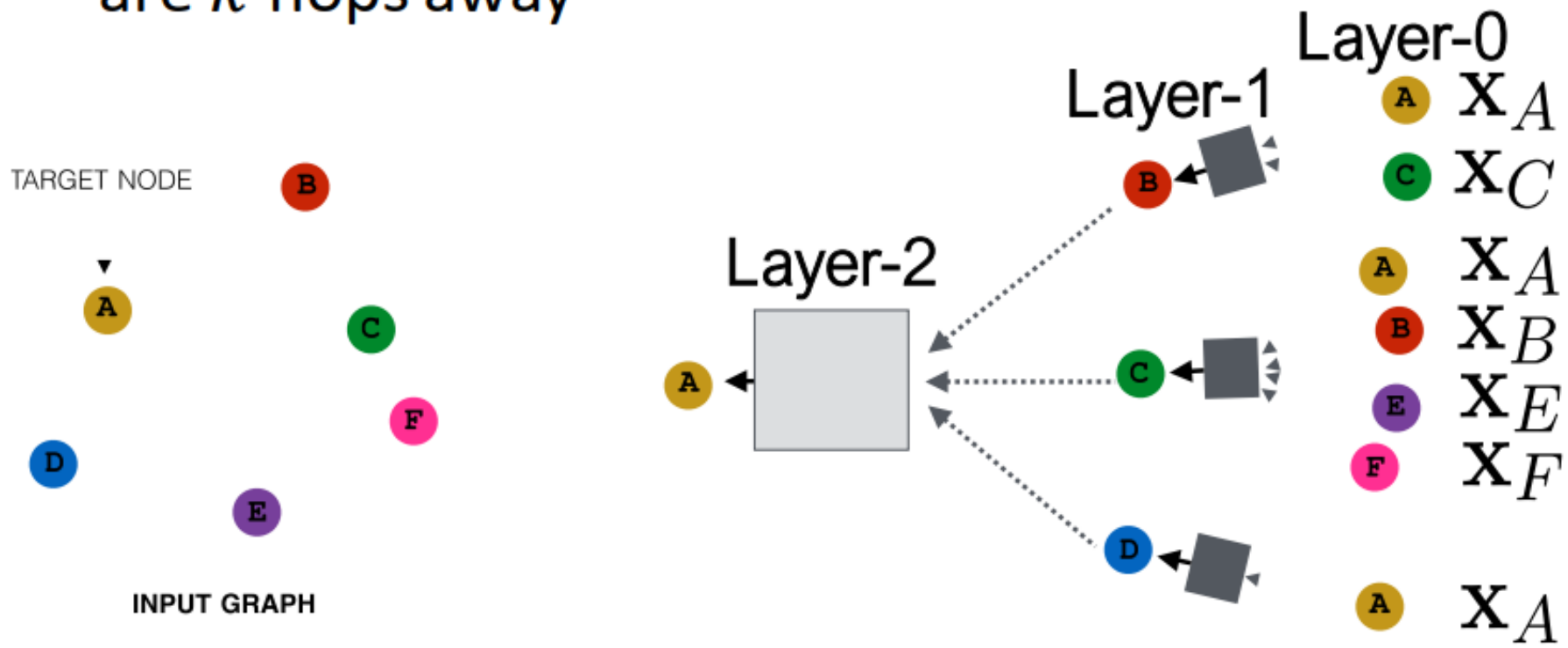
Every node defines a computation graph based on its neighborhood!



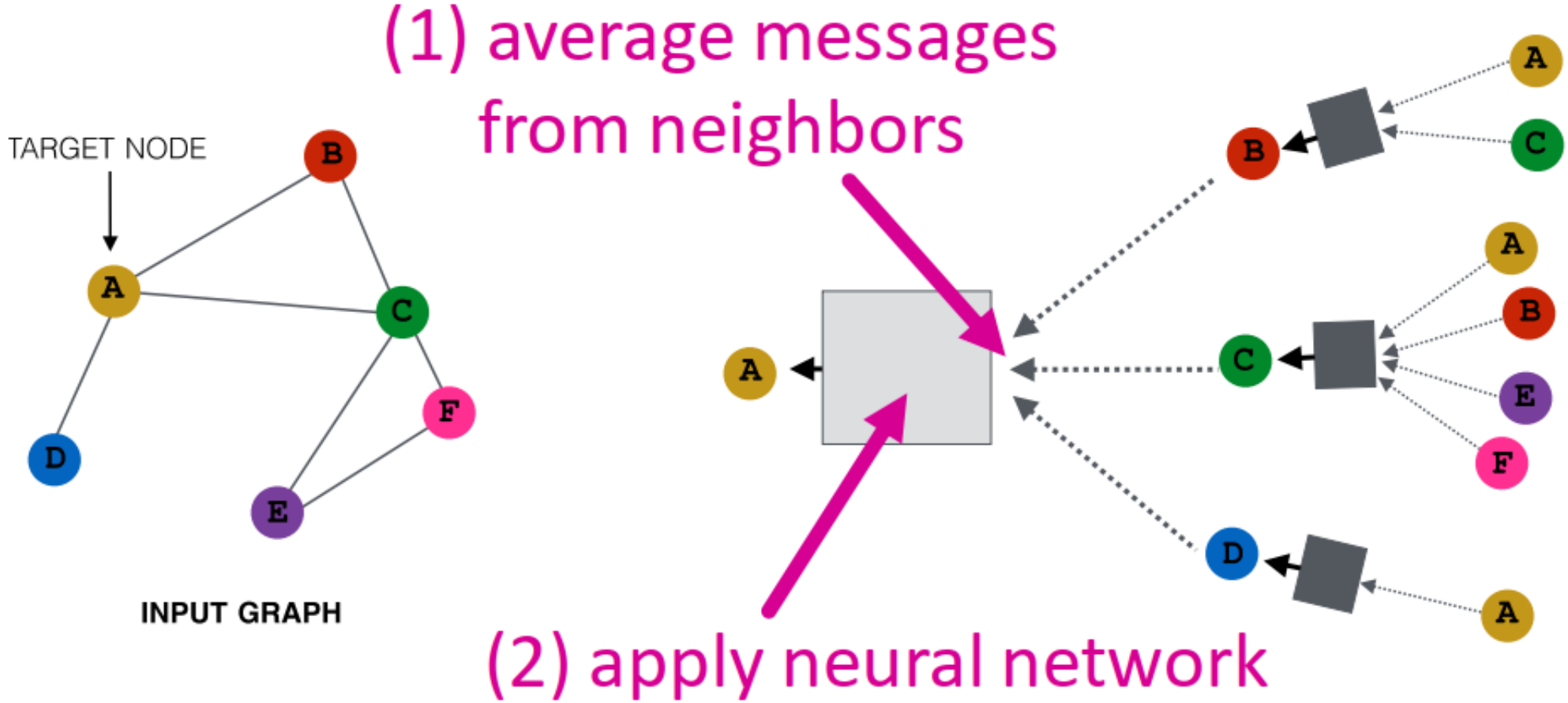
INPUT GRAPH



- Model can be of arbitrary depth:
  - Nodes have embeddings at each layer
  - Layer-0 embedding of node v is its input feature, x_v
  - Layer-k embedding gets information from nodes that are k hops away



- **Basic approach:** Average information from neighbors and apply a neural network



MPGNN (Message Passing GNN)

We begin here with the most basic GNN framework, which is a simplification of the original GNN models proposed by Merkwirth and Lengauer [2005] and Scarselli et al. [2009]

$$\mathrm{h}_u^{(k)} = \sigma\left(\mathrm{W}_{self}^{(k)}\mathrm{h}_u^{(k-1)} + \mathrm{W}_{neigh}^{(k)}\sum_{v\in\mathcal{N}(u)}\mathrm{h}_v^{(k-1)} + \mathrm{b}^{(k)}\right) \tag{7}$$

where,

$$\mathrm{W}_{self}^{(k)}, \mathrm{W}_{neigh}^{(k)} \in \mathbb{R}^{\mathrm{d}^{(k)}\times\mathrm{d}^{(k-1)}}$$
$$\mathrm{b}^{(k)} = \mathrm{b}_{self}^{(k)} + \mathrm{b}_{neigh}^{(k)}$$

In equation 7,

$\mathrm{W}_{self}^{(k)}$: weight matrix of target node's (u) multi-layer perceptron (MLP),

$\mathrm{W}_{neigh}^{(k)}$: weight matrix of MLP used with target node's neighborhood nodes $(\mathcal{N}(u))$,

$\sigma(.)$: activation function.

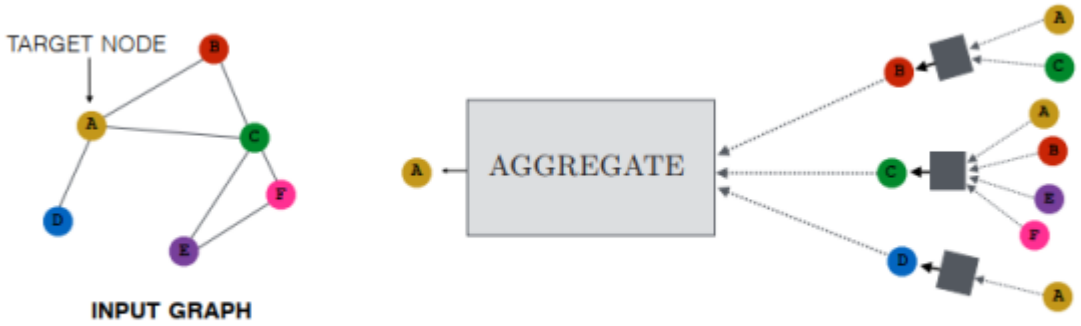


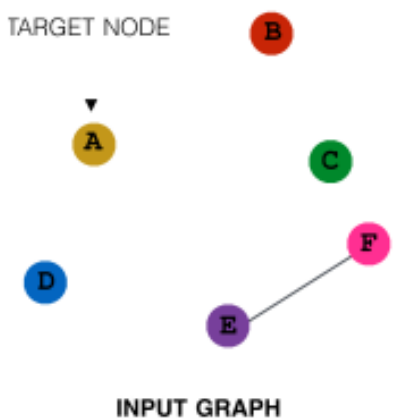
Fig. 1: GNN message passing mechanism

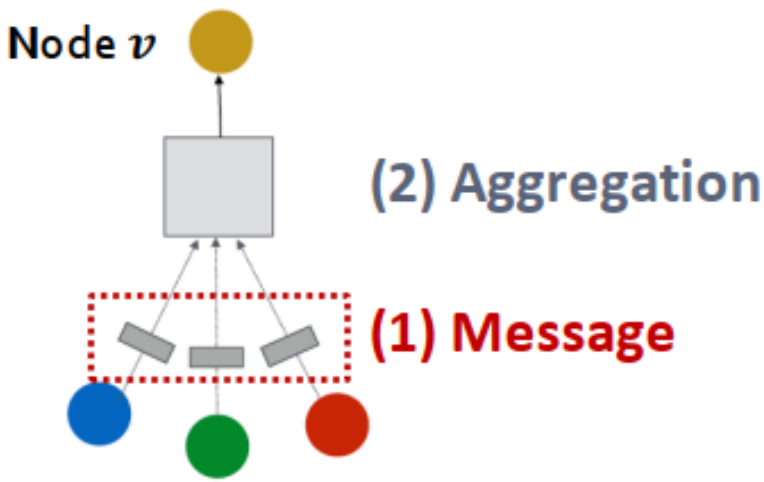
Message Passing

- Message Passing has been the key workforce of Graph Neural Networks, although there are work ongoing beyond Message Passing, it has proven to be effective.
- In English there is a famous saying, ***"A person is known by the company he keeps"***, message passing works similar to this. In a more technical term this called **Homophily,** which means the nodes connected to each other tend to have similar features and/or belong to similar classes.
- For example, in a node classification task, our Message Passing layer will take a source node, identify its neighbours, transform the neighbours features and finally pass it to the source node. This is process is parallelly done to all the nodes in the graph. Hence at the end of this step all the nodes are updated.
- Now lets look at what is discussed using an example.

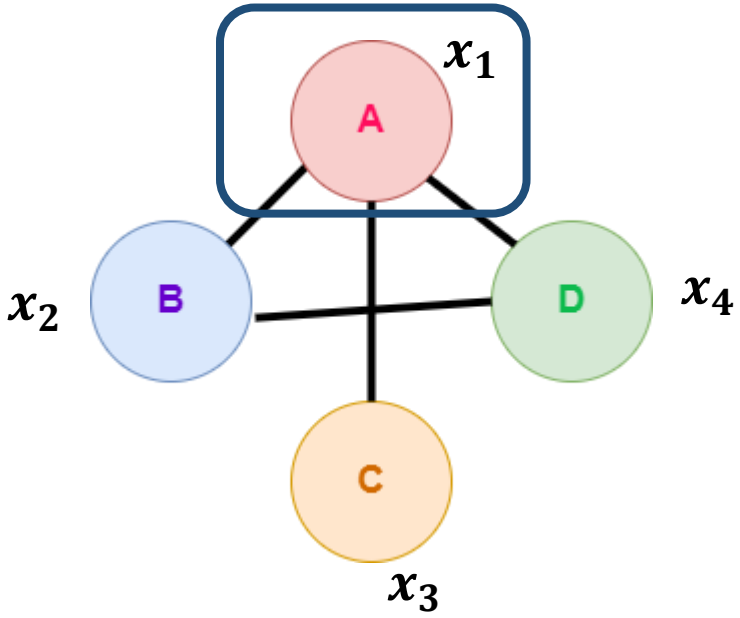
(1) Message computation

- **Message function**: $\mathbf{m}_u^{(l)} = \mathrm{MSG}^{(l)}\left(\mathbf{h}_u^{(l-1)}\right)$
  - **Intuition:** Each node will create a message, which will be sent to other nodes later
  - **Example:** A Linear layer $\mathbf{m}_u^{(l)} = \mathbf{W}^{(l)}\mathbf{h}_u^{(l-1)}$
    - Multiply node features with weight matrix $\mathbf{W}^{(l)}$

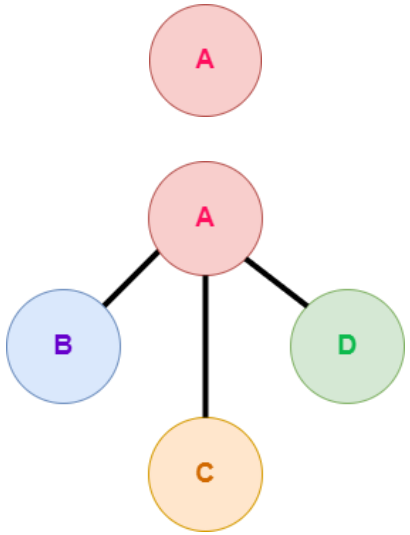


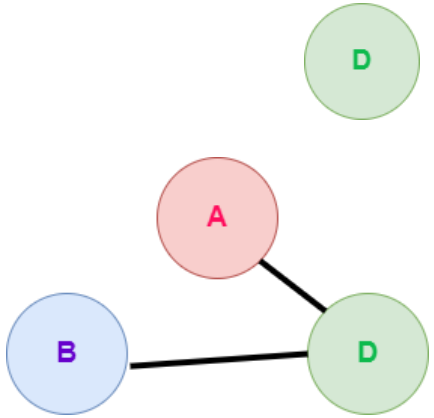


Message Passing Example

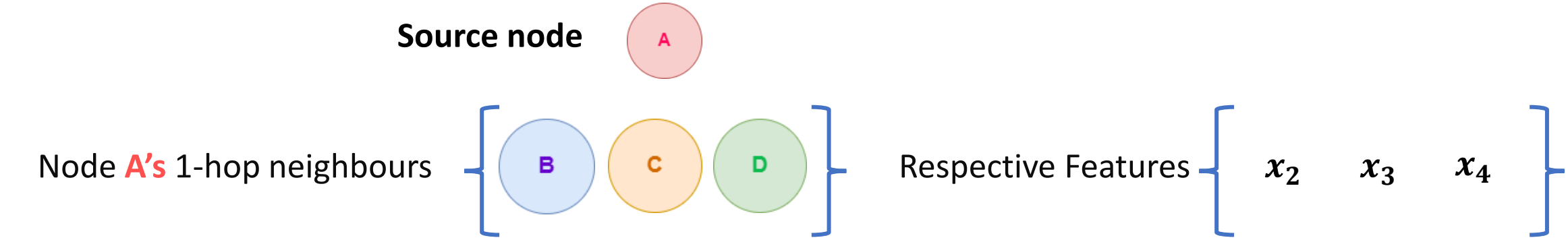


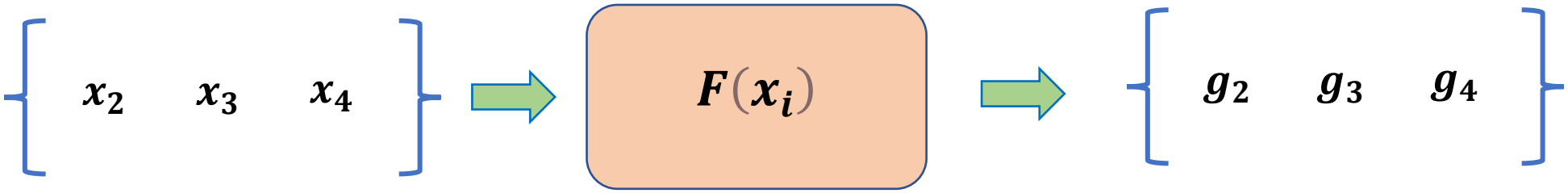
For example, lets identify the **1-hop neighbours** of the graph





Now let's take Node **A** as the source node and perform message passing

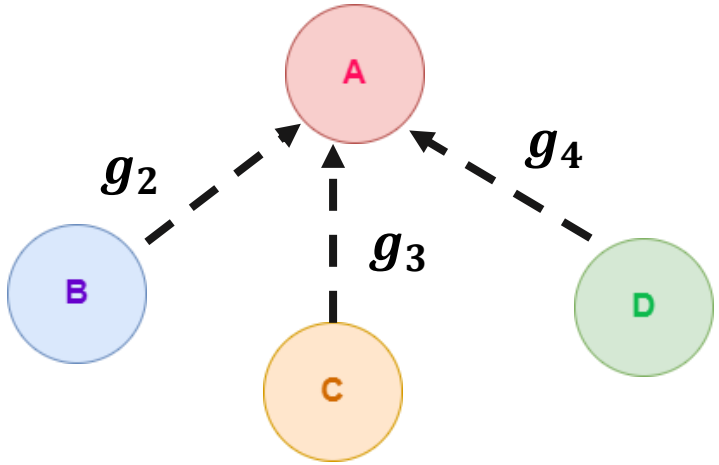




Here F can be MLP or RNN or simple affine function meaning simple linear tranformation will also work.
Hence,

$$F(x_i) = Wx_i + b \ (here\ we\ perform\ linear\ transformation\ on\ x_i)$$

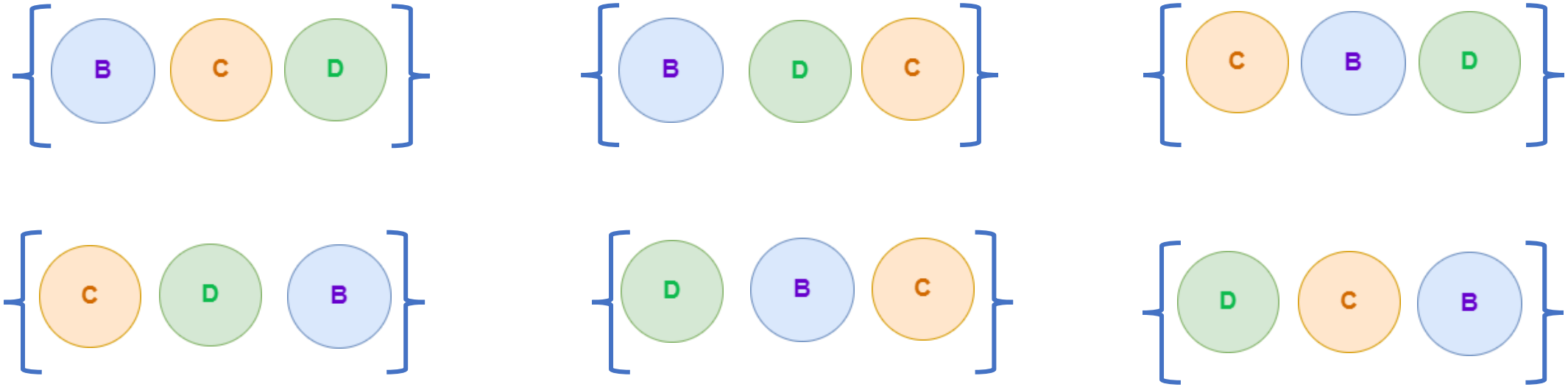
Finally, the transformed messages (g_i) gets passed to the source node



Aggregate

- Now that the messages from the 1-hop neighbour nodes have been passed into the source node.
- We have to now find a way to combine these messages together (AGGREGATE).
- But here is the catch, our aggregate function should be “**Permutation Invariant**”**.** This a key bias we will introduce to the GNNs. The justification is, that the nieghbour nodes should be taken in any order and shouldn’t affect the AGGREGATION.

Meaning, independent of the order we take the neighbours of the source node, the AGGREGATE function should produce the same results (Permutation Invariant)



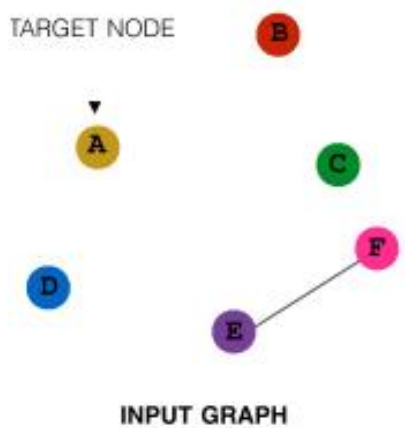
Nothing tells the first order of nodes is better the other combination of node order, this is the reason for Permutation Invariant Aggregation function selection.

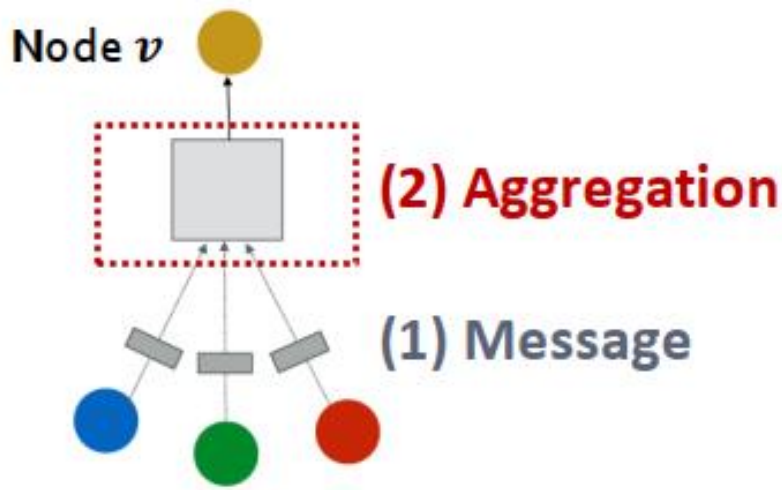
(2) Aggregation

- **Intuition:** Each node will aggregate the messages from node v's neighbors

$$\mathbf{h}_v^{(l)} = \mathrm{AGG}^{(l)}\left(\left\{\mathbf{m}_u^{(l)}, u \in N(v)\right\}\right)$$

- **Example:** $\mathrm{Sum}(\cdot)$, $\mathrm{Mean}(\cdot)$ or $\mathrm{Max}(\cdot)$ aggregator
  - $\mathbf{h}_v^{(l)} = \mathrm{Sum}(\{\mathbf{m}_u^{(l)}, u \in N(v)\})$





Some Famous Permutation Invariant Aggregation Functions

1. Summation ->1+2+3=3+2+1=2+1+3=6
2. Mean -> (1+2+3)/3=(3+2+1)/3=(2+1+3)=6/3=2
3. Min -> min(1,2,3)=min(3,2,1)=min(2,1,3) = 1
4. Max -> max(1,2,3)=max(3,2,1)=max(2,1,3) = 3

Hence let's define permutation invariant function AGG

$$m_i = AGG(g_j; j \in N_i)$$

m_i - Combined message of neighbours of I

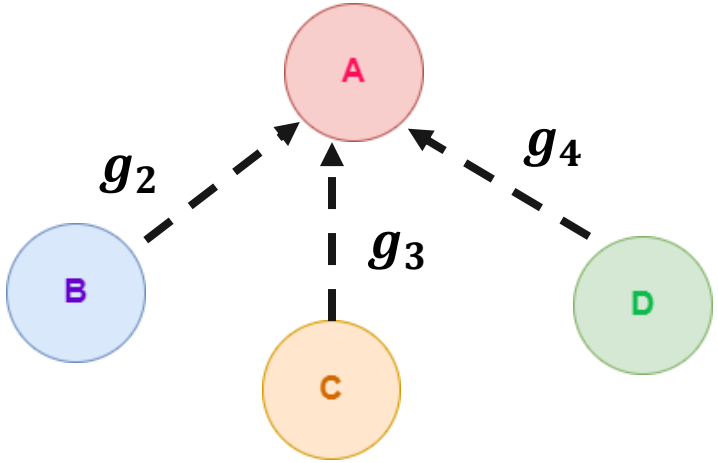
AGG – Can be any permutation invariant function (sum,mean,min ,max etc)

g_j - Transformed features of the neighbour nodes

N_i - Neigbourhood nodes of node i

To Recap

We perform message passing by transforming the features first.



Then we aggregate the messages by using a permutation invariant function

$$m_i = AGG(g_j; j \in N_i)$$

Update

- Finally, we need update the feature of the source node.
- Here we need to utilize the incoming aggregated message as well for the update.

Step 1

- We will first transform the feature of the source node using MLP or an affine function.

$$G(x_i)$$

Step 2

- Now we need to aggregate the transformed feature with m_i .
- To combine m_i and $G(x_i)$ normally in literature we use,
  1. Summation or Averaging
  2. Concatenation.

To elaborate

We can combine $\boldsymbol{m_i}$ and $\boldsymbol{G(x_i)}$ in the following way

$$(\boldsymbol{G(x_i)} + \boldsymbol{m_i})\ \boldsymbol{OR}\ (\boldsymbol{G(x_i)} \,||\, \boldsymbol{m_i}),$$

Here "||" stand for concatenation

NOTE:

- Few items to note, if summation is your choice of combining $\boldsymbol{m_i}$ and $\boldsymbol{G(x_i)}$ then dimensions of $\boldsymbol{m_i}$ and $\boldsymbol{G(x_i)}$ should agree.

Step 3

- Now that we have combined $\boldsymbol{m_i}$ and $\boldsymbol{G(x_i)}$ we will add the usual Neural Networks.
- By that, we will pass the combination of $\boldsymbol{m_i}$ and $\boldsymbol{G(x_i)}$ through the non-linear activation function.

$$\boldsymbol{h_i} = \boldsymbol{\sigma(G(x_i)}, \boldsymbol{m_i})$$

Where,

$\boldsymbol{\sigma}$ = Non-linearity (eg:sigmoid, ReLu, Tanh etc..)

- **Issue:** Information from node v itself **could get lost**
  - Computation of $\mathbf{h}_v^{(l)}$ does not directly depend on $\mathbf{h}_v^{(l-1)}$
- **Solution:** Include $\mathbf{h}_v^{(l-1)}$ when computing $\mathbf{h}_v^{(l)}$
  - **(1) Message: compute message from node v itself**
    - Usually, a **different message computation** will be performed

$\mathbf{m}_u^{(l)} = \mathbf{W}^{(l)}\mathbf{h}_u^{(l-1)}$

$\mathbf{m}_v^{(l)} = \mathbf{B}^{(l)}\mathbf{h}_v^{(l-1)}$

  - **(2) Aggregation:** After aggregating from neighbors, we can **aggregate the message from node v itself**
    - Via **concatenation** or **summation**

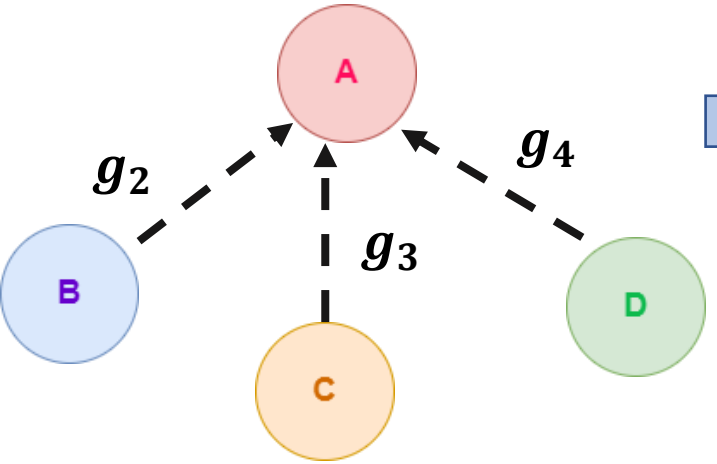
Then aggregate from node itself

$$\mathbf{h}_v^{(l)} = \text{CONCAT}\left(\text{AGG}\left(\left\{\mathbf{m}_u^{(l)}, u \in N(v)\right\}\right), \mathbf{m}_v^{(l)}\right)$$

First aggregate from neighbors

To sum up the operations in a GNN layer

Message Passing | **Aggregate** | **Update**



$$m_i = AGG(h_j; j \in N_i) \qquad h_i = \sigma(G(x_i), m_i)$$

Well, it's done, one layer of GNN has been formulated!!

Note:

All these steps are performed on all the nodes in the graph simultaneously.

Matrix formulation

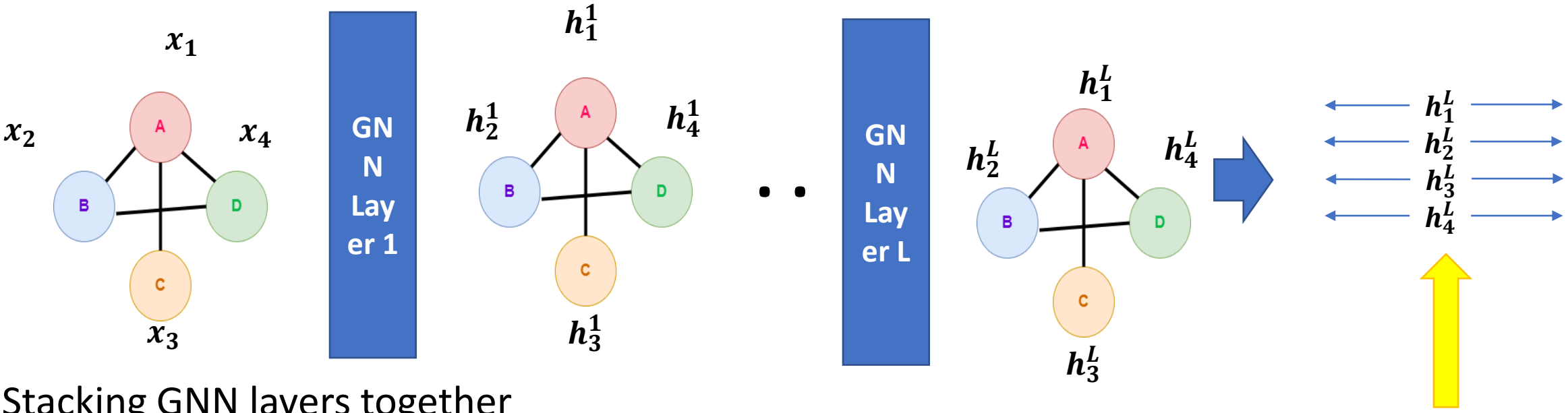
$$\mathrm{H}^{(k)} = \sigma\left(\mathrm{A}\mathrm{H}^{(k-1)}\mathrm{W}^{(k)}_{neigh} + \mathrm{H}^{(k-1)}\mathrm{W}^{(k)}_{self}\right) \quad (8)$$

where,

$$\mathrm{H}^{(k)} \in \mathbb{R}^{|\mathrm{v}|\times \mathrm{d}^{(k)}}$$

$$\mathrm{H}^{(k-1)} \in \mathbb{R}^{|\mathrm{v}|\times \mathrm{d}^{(k-1)}}$$

MPGNN can also be expressed in matrix format.



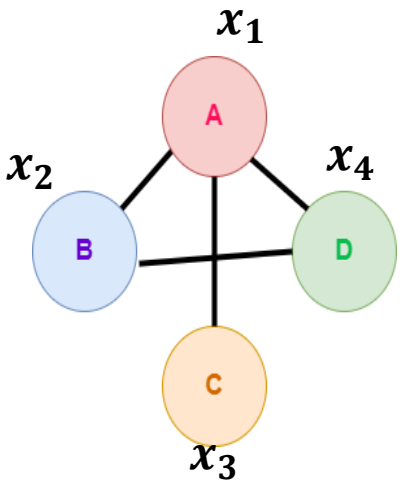
Stacking GNN layers together

Where h_i^l represents the embedding h of node i in the l^{th} layer.
i = 1,2,3,4
l = 1,2,3. . . L

Final learned representation for the nodes A,B,C,D. This can be used for the downstream tasks.

Graph Neural Networks as an End-to-End Solution

Input Graph

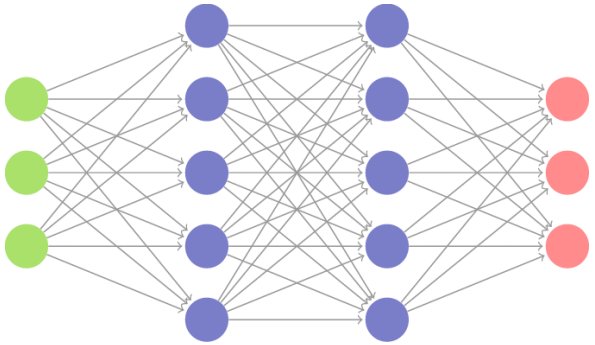


GNN Layers

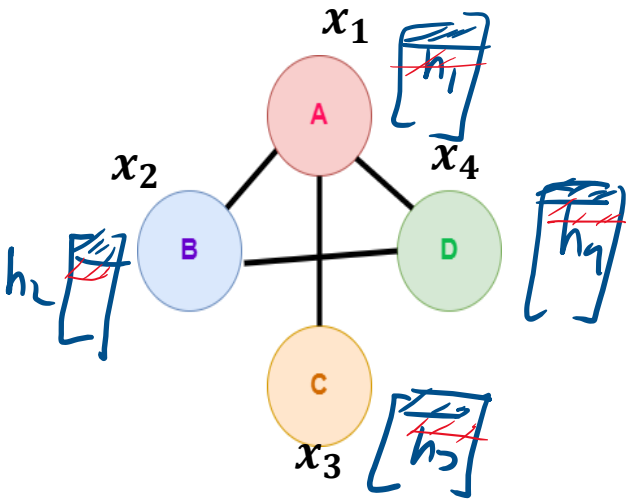
Learned representations

h_1^L
h_2^L
h_3^L
h_4^L

For downstream tasks like node classification, link prediction etc..



Achieving GNN layer operation using Adjacency Matrix (a simple approach)

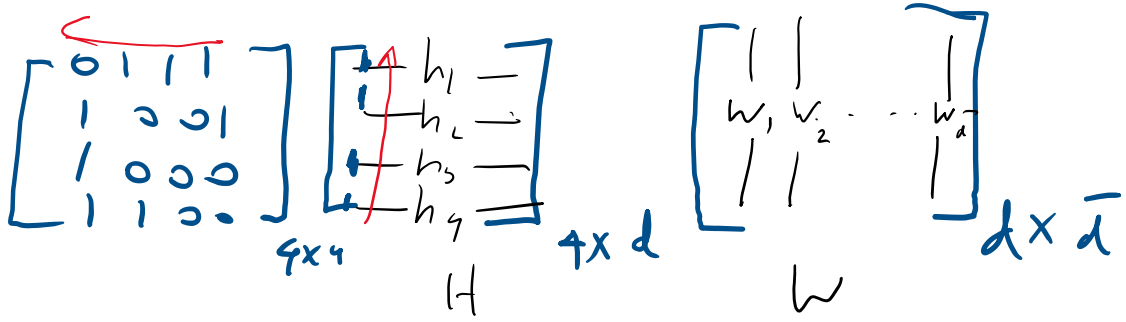


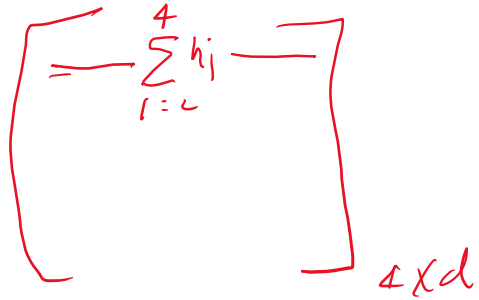
Graph G

	A	B	C	D
A	0	1	1	1
B	1	0	0	1
C	1	0	0	0
D	1	1	0	0

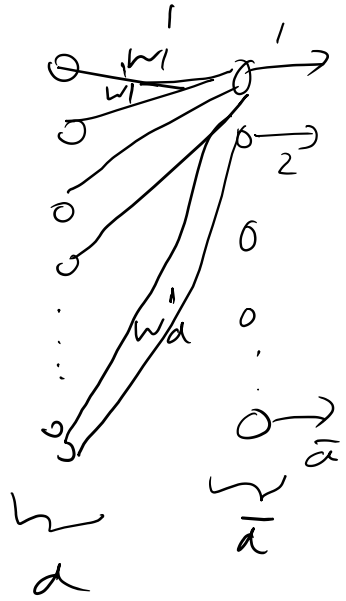
Adjacency matric for graph G

$$H^{(k)} = \sigma\left(AH^{(k-1)}W^{(k)}_{neigh} + H^{(k-1)}W^{(k)}_{self}\right)$$

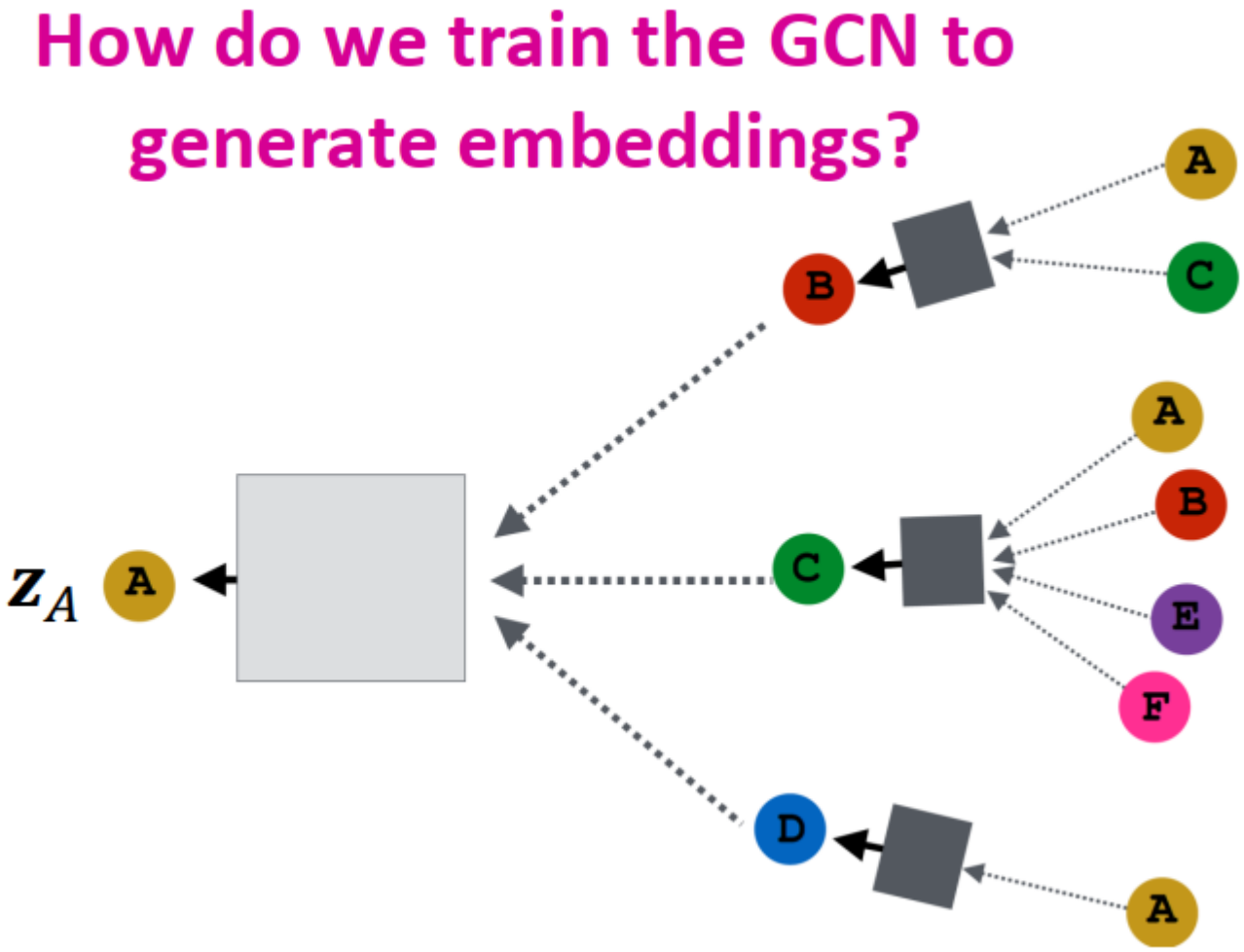




A H



Graph Convolutional Network (GCN)



Node level formulation

Trainable weight matrices
(i.e., what we learn)

$$\mathrm{h}_v^{(0)} = \mathrm{x}_v$$

$$\mathrm{h}_v^{(k+1)} = \sigma\left(\mathrm{W}_k \sum_{u \in \mathrm{N}(v)} \frac{\mathrm{h}_u^{(k)}}{|\mathrm{N}(v)|} + \mathrm{B}_k \mathrm{h}_v^{(k)}\right), \forall k \in \{0..K-1$$

$$\mathrm{z}_v = \mathrm{h}_v^{(K)}$$

Final node embedding

We can feed these **embeddings into any loss function** and run SGD to **train the weight parameters**

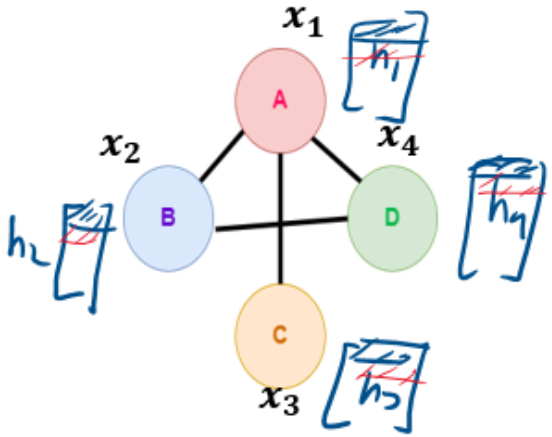
h_v^k: the hidden representation of node v at layer k

- W_k: weight matrix for neighborhood aggregation
- B_k: weight matrix for transforming hidden vector of self

Matrix level formulation

- **Many aggregations can b**
  efficiently by (sparse) ma
- Let $H^{(k)} = [h_1^{(k)} \dots h_{|V|}^{(k)}]^T$
- Then: $\sum_{u \in N_v} h_u^{(k)} = A_{v,:} H^{(k)}$
- Let D be diagonal matrix where
  $D_{v,v} = \text{Deg}(v) = |N(v)|$
  - The inverse of D: D^{-1} is also diagonal:
    $D_{v,v}^{-1} = 1/|N(v)|$
- **Therefore,**

$$\sum_{u \in N(v)} \frac{h_u^{(k-1)}}{|N(v)|} \Rightarrow H^{(k+1)} = D^{-1} A H^{(k)}$$

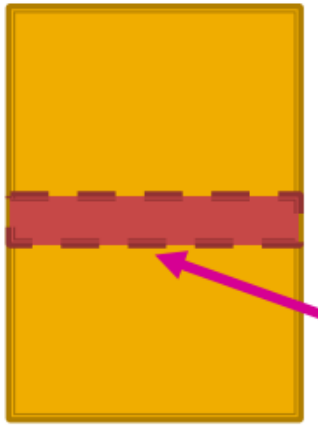


Graph G

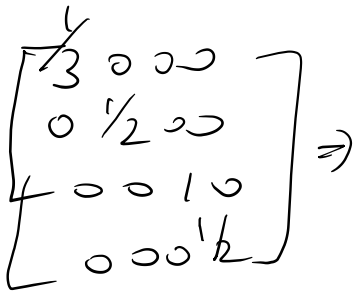
	A	B	C	D
A	0	1	1	1
B	1	0	0	1
C	1	0	0	0
D	1	1	0	0

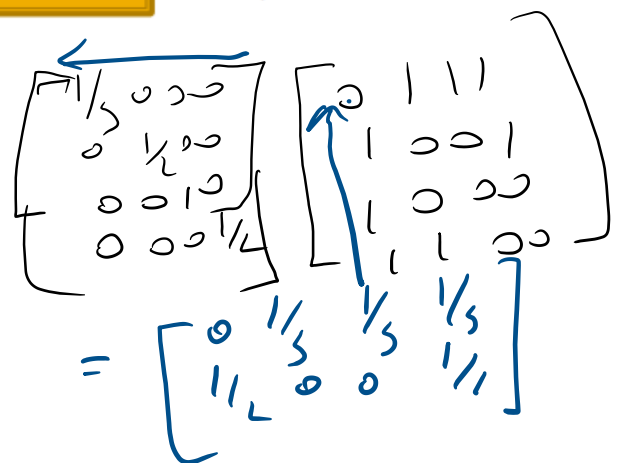
Adjacency matric for graph G

Matrix of hidden embeddings $H^{(k-1)}$



$h_i^{(k-1)}$

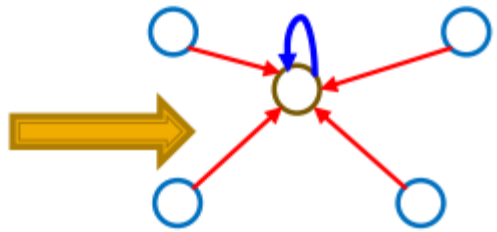
$D^{-1} =$ 



- Re-writing update function in matrix form:

$$H^{(k+1)} = \sigma(\tilde{A}H^{(k)}W_k^{\mathrm{T}} + H^{(k)}B_k^{\mathrm{T}})$$

where $\tilde{A} = D^{-1}A$



$$H^{(k)} = [h_1^{(k)} \dots h_{|V|}^{(k)}]^T$$

  - Red: neighborhood aggregation
  - Blue: self transformation

- In practice, this implies that efficient sparse matrix multiplication can be used ($\tilde{A}$ is sparse)

- **Note**: not all GNNs can be expressed in matrix form, when aggregation function is complex

- **(1) Graph Convolutional Networks (GCN)**

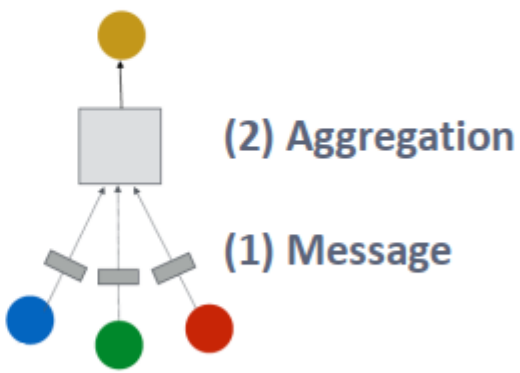
$$\mathbf{h}_v^{(l)} = \sigma\left(\mathbf{W}^{(l)} \sum_{u \in N(v)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|}\right)$$

- **How to write this as Message + Aggregation?**

Message

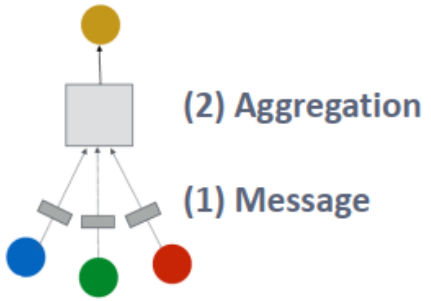
$$\mathbf{h}_v^{(l)} = \sigma\left(\sum_{u \in N(v)} \mathbf{W}^{(l)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|}\right)$$

Aggregation



(1) Graph Convolutional Networks (GCN)

$$\mathbf{h}_v^{(l)} = \sigma\left(\sum_{u \in N(v)} \mathbf{W}^{(l)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|}\right)$$



- **Message:**
  - Each Neighbor: $\mathbf{m}_u^{(l)} = \frac{1}{|N(v)|}\mathbf{W}^{(l)}\mathbf{h}_u^{(l-1)}$

Normalized by node degree
(In the GCN paper they use a slightly different normalization)

- **Aggregation:**
  - **Sum** over messages from neighbors, then apply activation
  - $\mathbf{h}_v^{(l)} = \sigma\left(\text{Sum}\left(\left\{\mathbf{m}_u^{(l)}, u \in N(v)\right\}\right)\right)$

In GCN graph is assumed to have self-edges that are included in the summation.

Original GCN paper introduced the renormalization method

$$\mathrm{m}_{\mathcal{N}(u)}^{k-1} = \sum_{v \in \mathcal{N}(u)} \frac{\mathrm{h}_v^{k-1}}{\sqrt{|\mathcal{N}(u)|\,|\mathcal{N}(v)|}}$$

$$\mathrm{H}^{(k)} = \sigma\left(\tilde{\mathrm{D}}^{-\frac{1}{2}}\tilde{\mathrm{A}}\tilde{\mathrm{D}}^{-\frac{1}{2}}\mathrm{H}^{(k-1)}\mathrm{W}^{(k)}\right)$$

$$\tilde{\mathrm{D}} = \mathrm{D} + \mathrm{I}$$

$$\tilde{\mathrm{A}} = \mathrm{A} + \mathrm{I}$$

GraphSAGE

- **(2) GraphSAGE**

$$\mathbf{h}_v^{(l)} = \sigma\left(\mathbf{W}^{(l)} \cdot \mathrm{CONCAT}\left(\mathbf{h}_v^{(l-1)}, \mathrm{AGG}\left(\left\{\mathbf{h}_u^{(l-1)}, \forall u \in N(v)\right\}\right)\right)\right)$$

- **How to write this as Message + Aggregation?**
  - **Message** is computed within the $\mathrm{AGG}(\cdot)$
  - **Two-stage aggregation**
    - **Stage 1:** Aggregate from node neighbors

$$\mathbf{h}_{N(v)}^{(l)} \leftarrow \mathrm{AGG}\left(\left\{\mathbf{h}_u^{(l-1)}, \forall u \in N(v)\right\}\right)$$

    - **Stage 2:** Further aggregate over the node itself

$$\mathbf{h}_v^{(l)} \leftarrow \sigma\left(\mathbf{W}^{(l)} \cdot \mathrm{CONCAT}(\mathbf{h}_v^{(l-1)}, \mathbf{h}_{N(v)}^{(l)})\right)$$

- **Mean:** Take a weighted average of neighbors

$$\text{AGG} = \sum_{u \in N(v)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|}$$

Aggregation **Message computation**

- **Pool:** Transform neighbor vectors and apply symmetric vector function $\text{Mean}(\cdot)$ or $\text{Max}(\cdot)$

$$\text{AGG} = \text{Mean}(\{\text{MLP}(\mathbf{h}_u^{(l-1)}), \forall u \in N(v)\})$$

Aggregation **Message computation**

- **LSTM:** Apply LSTM to reshuffled of neighbors

$$\text{AGG} = \text{LSTM}([\mathbf{h}_u^{(l-1)}, \forall u \in \pi(N(v))])$$

Aggregation

- **ℓ_2 Normalization:**
  - **Optional:** Apply ℓ_2 normalization to $\mathbf{h}_v^{(l)}$ at every layer
  - $\mathbf{h}_v^{(l)} \leftarrow \frac{\mathbf{h}_v^{(l)}}{\left\|\mathbf{h}_v^{(l)}\right\|_2} \forall v \in V$ where $\|u\|_2 = \sqrt{\sum_i u_i^2}$ (ℓ_2-norm)
  - Without ℓ_2 normalization, the embedding vectors have different scales (ℓ_2-norm) for vectors
  - In some cases (not always), normalization of embedding results in performance improvement
  - After ℓ_2 normalization, all vectors will have the same ℓ_2-norm

Graph Attention Network (GAT)

- **(3) Graph Attention Networks**

$$\mathbf{h}_v^{(l)} = \sigma\left(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}\right)$$

Attention weights

- **In GCN / GraphSAGE**
  - $\alpha_{vu} = \frac{1}{|N(v)|}$ is the **weighting factor (importance)** of node u's message to node v
  - $\Longrightarrow \alpha_{vu}$ is defined **explicitly** based on the structural properties of the graph (node degree)
  - $\Longrightarrow$ All neighbors $u \in N(v)$ are equally important to node v

(3) Graph Attention Networks

$$\mathbf{h}_v^{(l)} = \sigma\left(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}\right)$$

Attention weights

Not all node's neighbors are equally important

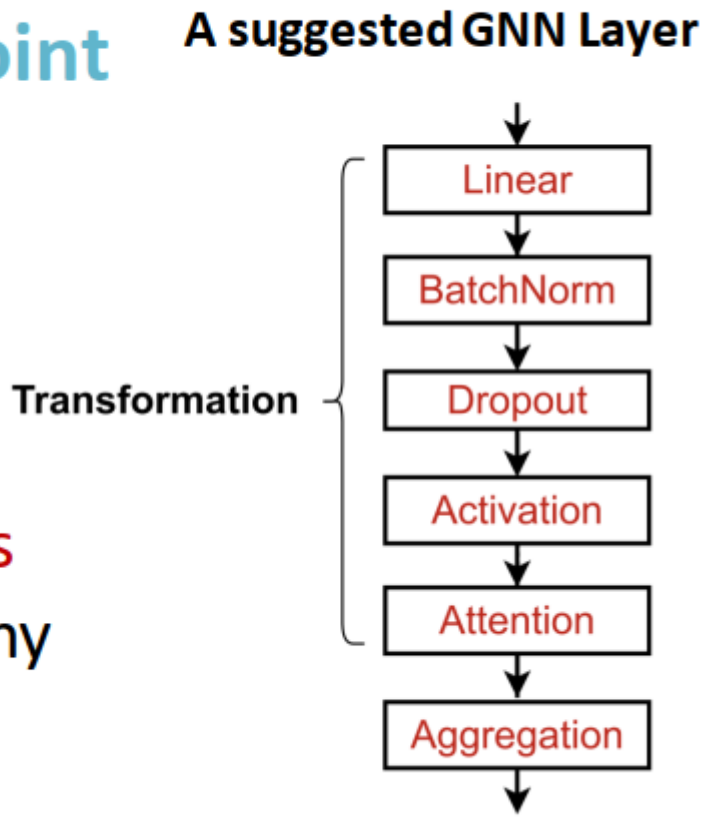
- **Attention** is inspired by cognitive attention.
- The **attention α_{vu}** focuses on the important parts of the input data and fades out the rest.
  - **Idea:** the NN should devote more computing power on that small but important part of the data.
  - Which part of the data is more important depends on the context and is learned through training.

$$\alpha_{u,v} = \frac{\exp\left(\mathrm{a}^{\mathrm{T}}\left[\mathrm{W}\mathrm{h}_u \parallel \mathrm{W}\mathrm{h}_v\right]\right)}{\sum_{w \in \mathcal{N}(u) \cup \{u\}} \exp\left(\mathrm{a}^{\mathrm{T}}\left[\mathrm{W}\mathrm{h}_u \parallel \mathrm{W}\mathrm{h}_w\right]\right)}$$

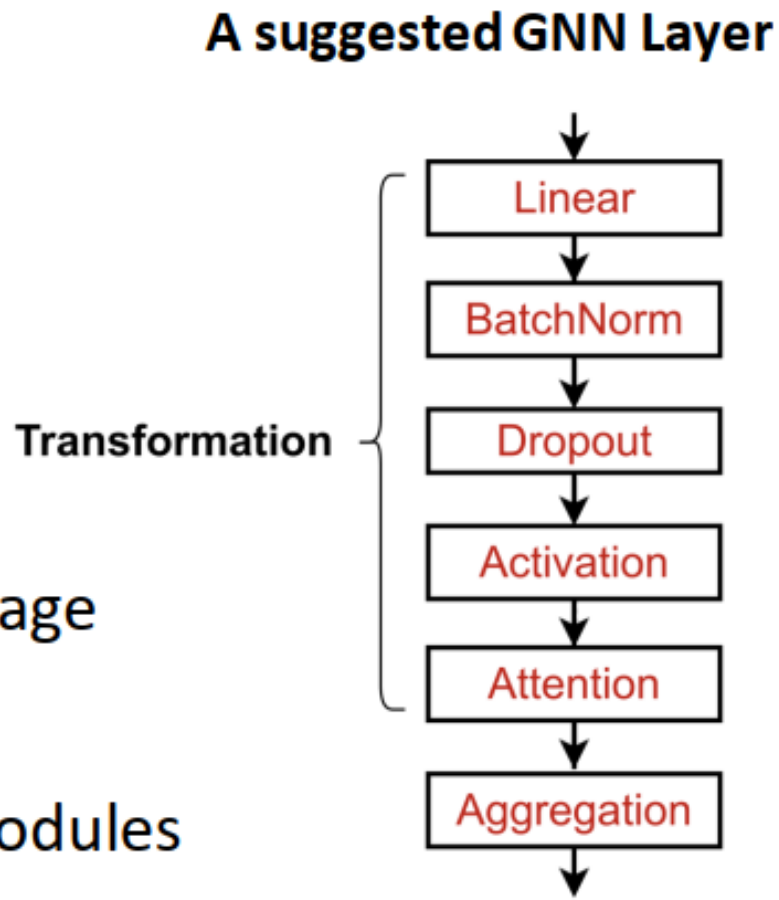
GNN Designing

GNN layers

- **In practice, these classic GNN layers are a great starting point**
  - We can often get better performance by considering a general GNN layer design
  - Concretely, we can include modern deep learning modules that proved to be useful in many domains



- **Many modern deep learning modules can be incorporated into a GNN layer**
  - **Batch Normalization:**
    - Stabilize neural network training
  - **Dropout:**
    - Prevent overfitting
  - **Attention/Gating:**
    - Control the importance of a message
  - **More:**
    - Any other useful deep learning modules

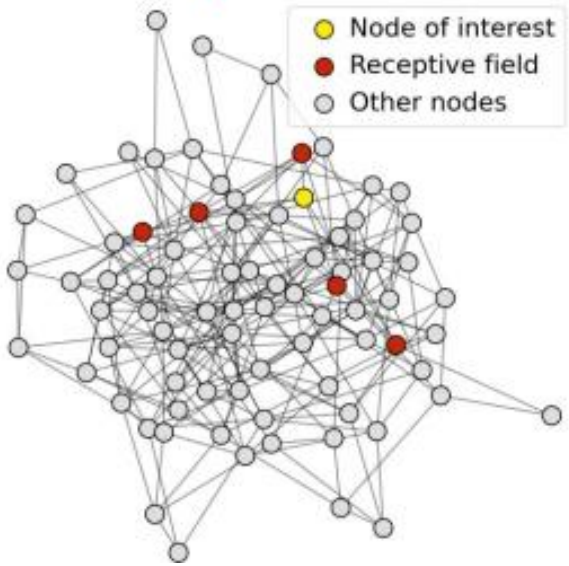


Over-smoothing effect

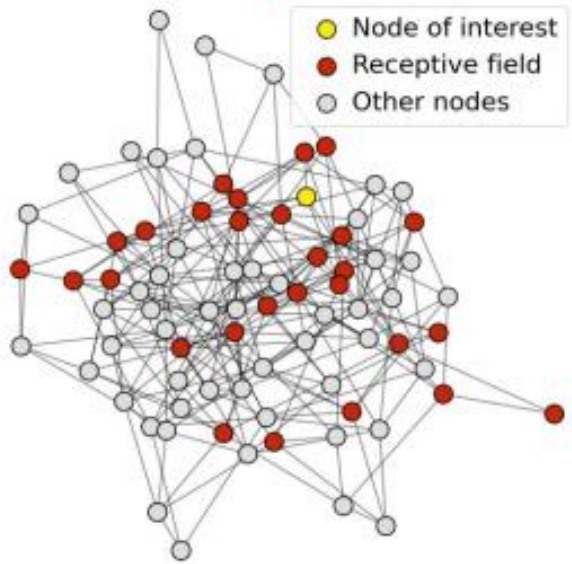
- **The Issue of stacking many GNN layers**
  - GNN suffers from **the over-smoothing problem**
- **The over-smoothing problem: all the node embeddings converge to the same value**
  - This is bad because we **want to use node embeddings to differentiate nodes**
- **Why does the over-smoothing problem happen?**

- **Receptive field:** the set of nodes that determine the embedding of a node of interest
  - **In a K-layer GNN, each node has a receptive field of K-hop neighborhood**

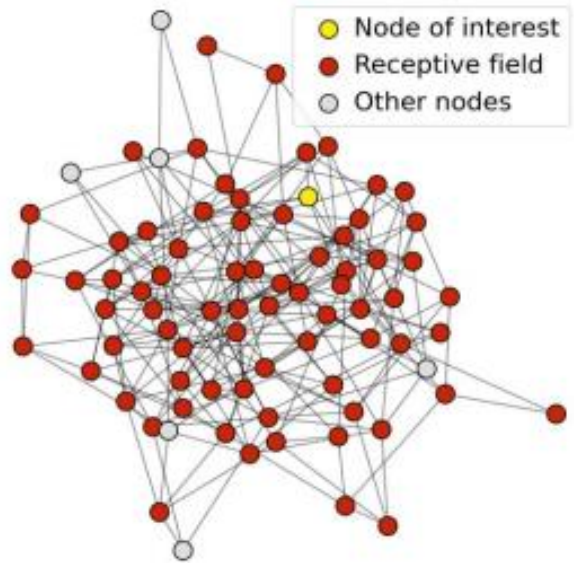
Receptive field for 1-layer GNN



Receptive field for 2-layer GNN



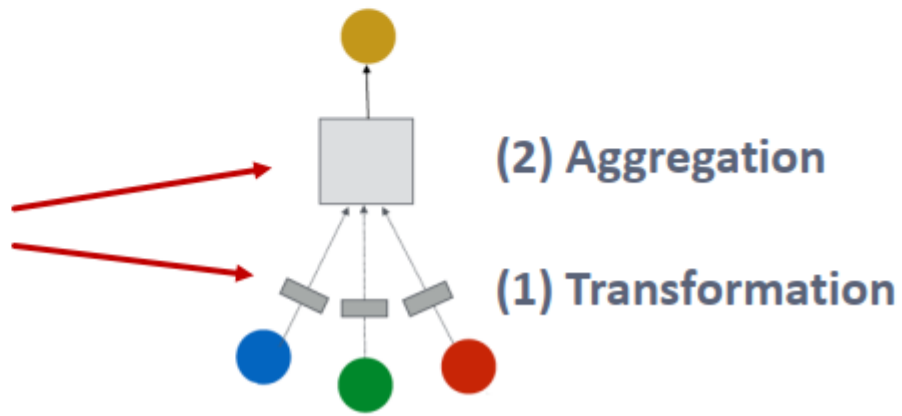
Receptive field for 3-layer GNN



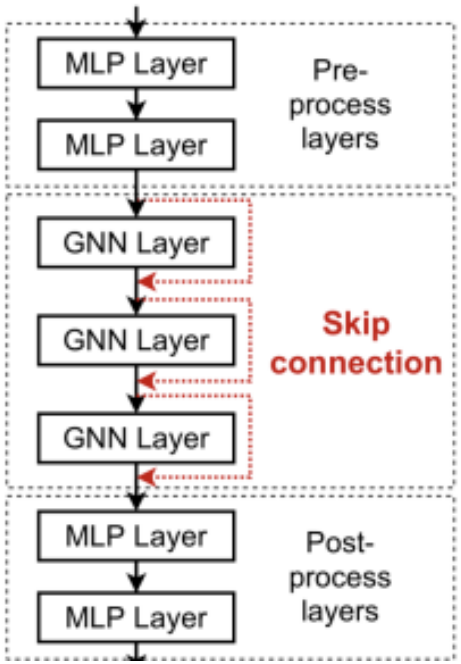
- **We can explain over-smoothing via the notion of receptive field**
  - We knew **the embedding of a node is determined by its receptive field**
    - If two nodes **have highly-overlapped receptive fields, then their embeddings are highly similar**
  - **Stack many GNN layers → nodes will have highly-overlapped receptive fields → node embeddings will be highly similar → suffer from the over-smoothing problem**
- **Next:** how do we overcome over-smoothing problem?

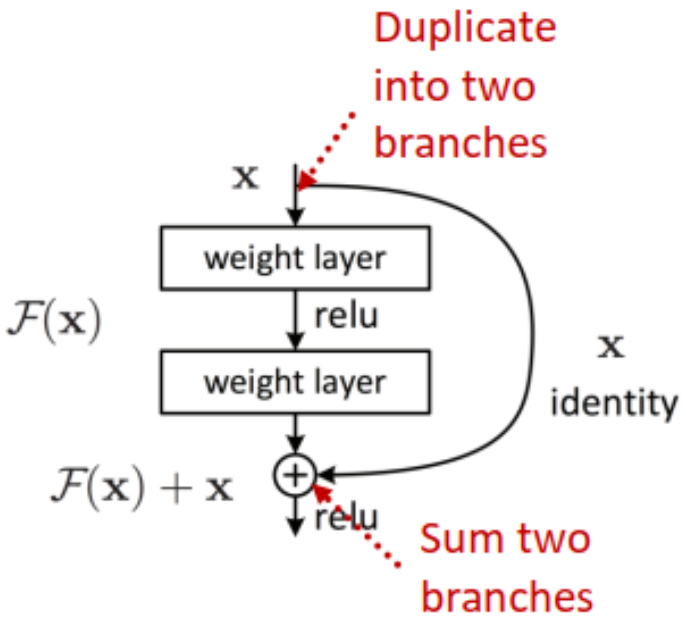
- **Solution 1:** Increase the expressive power **within each GNN layer**
  - In our previous examples, each transformation or aggregation function only include one linear layer
  - We can **make aggregation / transformation become a deep neural network**!

If needed, each box could include a 3-layer MLP



- **Lesson 2: Add skip connections in GNNs**
  - **Observation from over-smoothing:** Node embeddings in earlier GNN layers can sometimes better differentiate nodes
  - **Solution:** We can increase the impact of earlier layers on the final node embeddings, **by adding shortcuts in GNN**





Idea of skip connections:

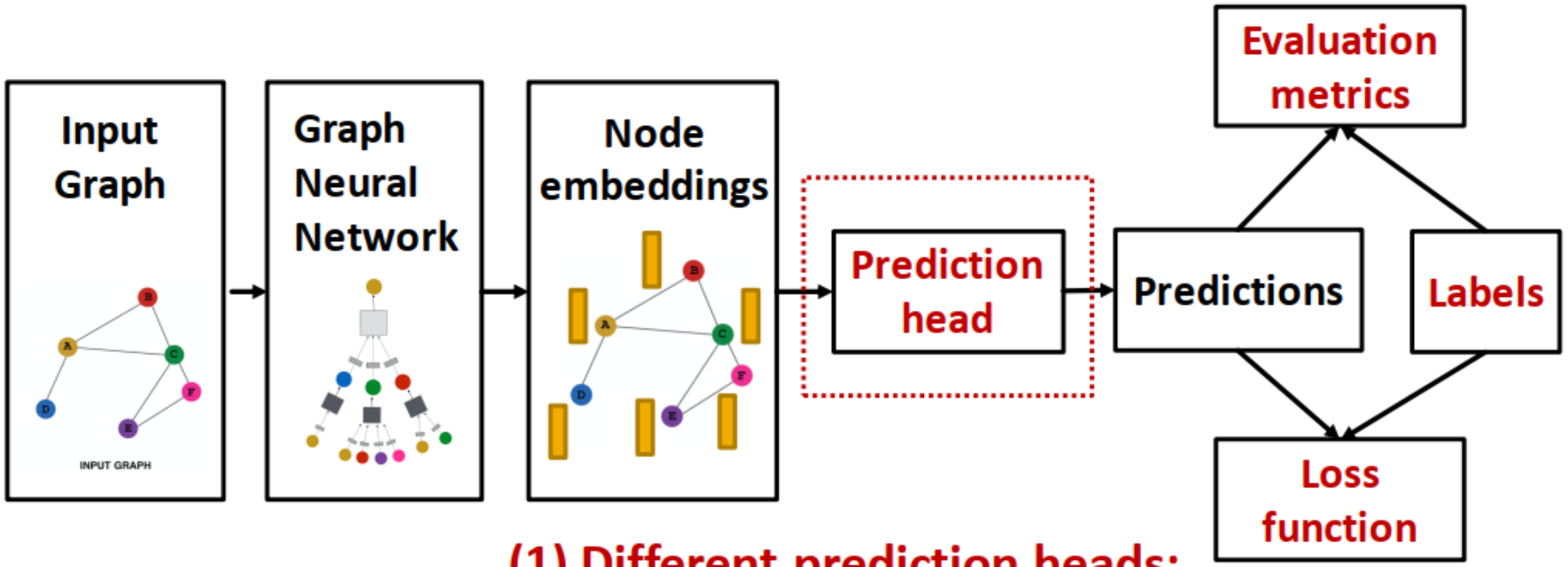
Before adding shortcuts:

$$\boldsymbol{F}(\mathbf{x})$$

After adding shortcuts:

$$\boldsymbol{F}(\mathbf{x}) + \mathbf{x}$$

GNN Training Pipeline

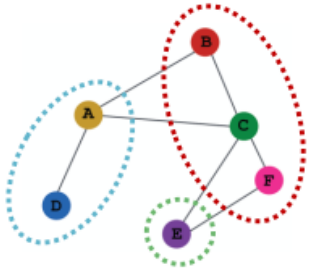


(1) Different prediction heads:

- **Node-level tasks**
- **Edge-level tasks**
- **Graph-level tasks**

Transductive/Inductive training example

- **Transductive** node classification
  - **All the splits can observe the entire graph structure**, but can only observe the labels of their respective nodes

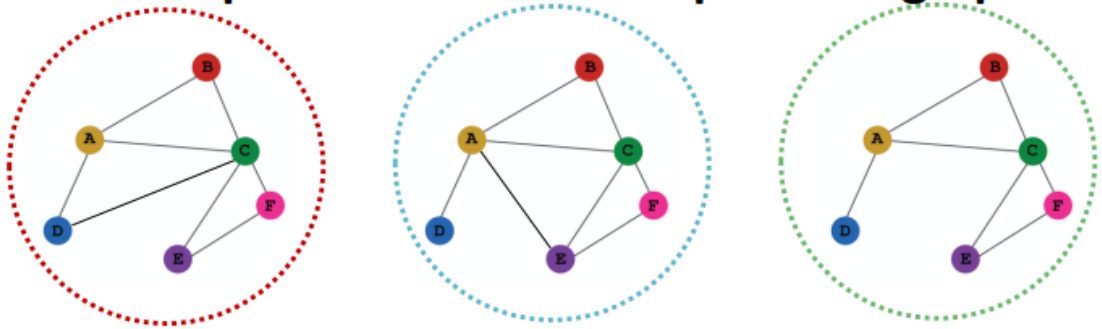


Training

Validation

Test

- **Inductive** node classification
  - Suppose we have a dataset of 3 graphs
  - **Each split contains an independent graph**



Training

Validation

Test

References

1. W. L. Hamilton, Graph Representation Learning Textbook.
   (freely available : https://www.cs.mcgill.ca/~wlh/grl_book/)
2. CS224: Machine Learning with Graphs 2021, Lecture Notes (link: http://web.stanford.edu/class/cs224w/)
3. Petar Velickovic, Theoretical Foundations of Graph Neural Networks
   (link: https://www.youtube.com/watch?v=uF53xsT7mjc&t=2113s)
4. Deep Graph Library Tutorials (link: https://docs.dgl.ai/guide/index.html)
5. PyTorch Geometric Tutorials
   (link: https://pytorch-geometric.readthedocs.io/en/latest/notes/colabs.html)
6. Aleksa Gordic, GNN research paper explanations
   (link: https://www.youtube.com/playlist?list=PLBoQnSflObckArGNhOcNg7lQG_f0ZlHF5)
7. Learning on Graphs Conference, GNN oriented conference (link: https://logconference.org/)
8. LoGaG: Learning on Graphs and Geometry Reading Group (link: https://hannes-stark.com/logag-reading-group)